



MALLA REDDY TECHNICAL CAMPUS
A Constituent Unit of MALLA REDDY VISHWAVIDYAPEETH
Deemed to be University
Approved by UGC & AICTE New Delhi
Maisammaguda, Medchal (Dist), Hyderabad - 500100, Telangana



DESIGN AND ANALYSIS OF ALGORITHMS



QUESTION BANK



MODULE – I

S.No	Short Answer Questions
1	Define an Algorithm.
2	Explain the characteristics of an Algorithm.
3	What is the difference between Time Complexity and Space Complexity?
4	What is the Divide and Conquer Technique?
5	How many multiplications are required in Strassen's algorithm?
6	What is a pivot element in Quick Sort?
7	What is Performance Analysis of an Algorithm?
8	What is Merge Sort?
9	What is Binary Search?
10	What is the main advantage of Divide and Conquer algorithms?

S.No	Long Answer Questions
1	Explain the Tower of Hanoi problem in detail. Write the recursive Algorithm for solving it and describe the steps involved in moving the disks from source to destination.
2	Explain performance analysis of algorithms. Discuss time complexity and space complexity with suitable algorithm examples.
3	What is Binary Search? Explain the algorithm with a suitable example and analyze its time complexity.
4	Explain Strassen's Matrix Multiplication with an Example.
5	Explain the Divide and Conquer strategy with an Algorithm and discuss its applications.
6	Explain Merge sort Algorithm with an example and analyze its time complexity.
7	Define time complexity. Explain asymptotic notations such as Big-O, Omega (Ω), and Theta (Θ).



8	What is Algorithm Analysis? Explain Algorithm Specification and describe the different methods of representing an algorithm. Discuss the various Algorithm Design Strategies with suitable examples.
9	Compare Quick Sort and Merge Sort based on their working principle, And mention their time complexity, space complexities.
10	Explain the Divide and Conquer strategy in detail. Discuss its three fundamental steps with a suitable example.



MODULE – II

S.No	Short Answer Questions
1	What is Hamiltonian cycle?
2	Write an Algorithm for Simple Union Operation?
3	What is the state-space tree in backtracking?
4	Define the Find operation in a disjoint-set data structure.
5	How is a Heap Implemented?
6	What is the Sum of Subsets Problem?
7	Define Max Heap and Min Heap
8	Write the general algorithm for Backtracking.
9	What is the Sum of Subsets Problem?
10	What is the Graph coloring?

S.No	Long Answer Questions
1	Explain Disjoint Set Operations. Describe Union and Find algorithms with examples.
2	Explain the N-Queens Problem using Backtracking. Illustrate the solution for a 4-Queens problem.
3	Describe the general method of Backtracking with an algorithm and write its applications.



4	Generate the state space tree and find the solutions for the subset sum for $N=4$, $M=50$, $W=\{10,20,30,40\}$ using backtracking approach.
5	Explain the Graph Coloring problem and solve it using the Backtracking technique.
6	What is a Hamiltonian Cycle? Explain the Backtracking solution for finding Hamiltonian Cycles in a graph.
7	Explain the Heap data structure. Perform Heap Sort on the following elements and obtain the sorted order. Show all intermediate steps. 2, 9, 7, 6, 5, 8
8	Explain the Disjoint Set data structure in detail. Describe its operations with suitable examples.
9	Explain the general method of Backtracking. Discuss its state-space tree in detail.
10	Compare the applications of Backtracking for Sum of Subsets and Graph Coloring problems.



MODULE – III

S.No	Short Answer Questions
1	What is Dynamic Programming?
2	What are the two important properties of Dynamic Programming?
3	List applications of Dynamic Programming.
4	What is memoization?
5	What is tabulation?
6	What is an Optimal Binary Search Tree?
7	What is the objective of the 0/1 Knapsack problem?
8	What is the All Pairs Shortest Path problem?
9	What is the Traveling Salesperson Problem (TSP)?
10	What is Reliability Design?

S.No	Long Answer Questions
1	Explain the Dynamic Programming technique in detail. Discuss its general method and the principle of optimality.
2	Differentiate between Dynamic Programming and Divide and Conquer.
3	Explain the control abstraction of Dynamic Programming. Write the general control abstraction Algorithm.
4	Consider a 0/1 Knapsack Problem with $n = 4$ items having weights $w_1=2$, $w_2=3$, $w_3=4$, and $w_4=5$ and corresponding profits $p_1=3$, $p_2=4$, $p_3=5$, and $p_4=6$. If the knapsack capacity is $W = 5$, determine the maximum profit and the selected items.
5	Solve a Reliability Design problem using Dynamic Programming for a system with multiple stages and alternative component choices and determine the design with maximum reliability.



6	Compute All pairs shortest path problem for following graph
7	Solve the Travelling sales Persons problem using Dynamic Programming. and determine the minimum-cost tour for below given graph.
8	What are overlapping subproblems and optimal substructure? Explain these properties in detail and discuss their importance in Dynamic Programming.
9	Compute All pairs shortest path problem for following graph
10	Explain the working of the Floyd–Warshall algorithm with a suitable example.



MODULE – IV

S.No	Short Answer Questions
1	What is Greedy method?
2	Write any two characteristics of Greedy Algorithm.
3	List the applications of Greedy Method.
4	Define spanning tree.?
5	State the Job-Sequencing Deadline Problem.
6	What is the importance of fractional knapsack algorithm in our daily life?
7	Find an optimal solution to the knapsack instance $n = 4$ objects and capacity $m = 15$, with profits (10, 5, 7, 11) and weights (3, 4, 3, 5).
8	Define Single Source Shortest Path Problem?
9	Mention the types of Binary Tree Traversal.
10	Define minimum spanning tree.

S.No	Long Answer Questions																								
1	Explain the terms feasible solution, optimal solution and objective function with suitable examples.																								
2	Find the solution generated by the Job Sequencing with Deadlines problem for N=7 jobs. The profits and deadlines are given in the following table: <table><tr><td>Job</td><td>J1</td><td>J2</td><td>J3</td><td>J4</td><td>J5</td><td>J6</td><td>J7</td></tr><tr><td>Profit</td><td>3</td><td>5</td><td>20</td><td>18</td><td>1</td><td>6</td><td>30</td></tr><tr><td>Deadline</td><td>1</td><td>3</td><td>4</td><td>3</td><td>2</td><td>1</td><td>2</td></tr></table>	Job	J1	J2	J3	J4	J5	J6	J7	Profit	3	5	20	18	1	6	30	Deadline	1	3	4	3	2	1	2
Job	J1	J2	J3	J4	J5	J6	J7																		
Profit	3	5	20	18	1	6	30																		
Deadline	1	3	4	3	2	1	2																		



3	Item	P_i(Profit)	W_i(Weight)
	I_1	25	18
	I_2	24	15
	I_3	15	10
Consider the above Fractional Knapsack instance with $N=3$, $M=20$. Find the optimal solution using Greedy method.			
4	Consider the following weighted graph . Find the Single Source Shortest Paths from the given source vertex using the Matrix Method (Dijkstra's Algorithm), and determine the shortest distance from the source vertex to each of the remaining vertices.		
5	Write and explain Prim's algorithm for finding the Minimum Cost Spanning Tree of a connected graph. Execute the algorithm on the given graph.		
6	Generate the Minimum Cost Spanning Tree (MCST) for the given graph by applying Kruskal's algorithm.		



7	Explain the various tree traversal techniques for binary trees with suitable examples.
8	Differentiate between Breadth First Search (BFS) and Depth First Search (DFS) with suitable examples.
9	Explain Connected Components and Bi-connected Components in graphs with suitable examples.
10	Define a Spanning Tree. Explain its properties and discuss the methods for finding a spanning tree of a connected graph with a suitable example.



MODULE – V

S.No	Short Answer Questions
1	What is Branch and Bound?
2	What is the Travelling Salesperson Problem (TSP)?
3	What is the 0/1 Knapsack Problem?
4	What is LC Branch and Bound?
5	What is FIFO Branch and Bound?
6	What is an NP-Hard problem?
7	What is an NP-Complete problem?
8	What is a non-deterministic algorithm?
9	What is Cook's Theorem?
10	Differentiate between NP-Hard and NP-Complete problems.

S.No	Long Answer Questions
1	Explain the Branch and Bound technique in detail. Discuss its general method, basic terminology, state-space tree, branching, bounding function, and pruning with a suitable example.
2	Define the term branch and bound. What are the methods of branch and bound?
3	Explain the basic concepts of P, NP, NP-Hard, and NP-Complete classes. Discuss the relationship between these classes with suitable examples.
4	What are non-deterministic algorithms? Compare deterministic and non-deterministic algorithms.



5	Compare LC Branch and Bound and FIFO Branch and Bound.
6	Explain Cook's Theorem (Cook-Levin Theorem) in detail. Discuss its significance in computational complexity theory
7	What are NP-hard and NP-Complete Problems? Explain their Characteristics.
8	Explain FIFO Branch and Bound with an example.
9	Explain the general Branch and Bound algorithm. Discuss how live nodes, E-nodes, and dead nodes are handled during the search.
10	Compare Backtracking and Branch and Bound.



MODULE – I

Short Answer Questions

1. Define an Algorithm.

An Algorithm is a finite sequence of instructions, each of which has a clear meaning and can be performed with a finite amount of effort in a finite length of time.

2. Explain the characteristics of an Algorithm.

Input: there are zero or more quantities, which are externally supplied;

Output: at least one quantity is produced

Definiteness: each instruction must be clear and unambiguous;

Finiteness: if we trace out the instructions of an algorithm, then for all cases the algorithm will terminate after a finite number of steps;

Effectiveness: every instruction must be sufficiently basic that it can in principle be carried out by a person using only pencil and paper.

3. What is the difference between Time Complexity and Space Complexity?

Time complexity measures how much time an algorithm takes based on input size. Space complexity measures how much memory an algorithm uses. Both help evaluate and compare algorithm efficiency.

4. What is the Divide and Conquer Technique?

Divide and Conquer is an algorithm design technique in which a problem is repeatedly divided into smaller subproblems of the same type, each subproblem is solved independently, and their solutions are combined to obtain the final solution.

5. How many multiplications are required in Strassen's algorithm?

In conventional matrix multiplication of two 2×2 matrices, 8 multiplications are required.

In Strassen's Matrix Multiplication algorithm, the number of multiplications are reduced to 7 multiplications and 18 additions/subtractions.



6. What is a pivot element in Quick Sort?

A pivot element is the element selected from the array during the Quick Sort process to partition the array into two parts.

1. Elements smaller than the pivot are placed on its left side.
2. Elements greater than the pivot are placed on its right side.
3. After partitioning, the pivot is placed in its correct sorted position.

7. What is Performance Analysis of an Algorithm?

Performance Analysis is the process of evaluating an algorithm based on the time and memory required for its execution.

Time Complexity: Measures the execution time.

Space Complexity: Measures the memory required.

8. What is Merge Sort?

Merge Sort is a Divide and Conquer sorting algorithm that divides an array into smaller subarrays, recursively sorts them, and then merges the sorted subarrays.

Best, Average and Worst Case: $O(n \log n)$

Space Complexity: $O(n)$

9. What is Binary Search?

Binary Search is a searching technique used to find an element in a sorted array by repeatedly dividing the search interval into two halves.

Compare the key with the middle element.

If the key is smaller, search the left half.

If the key is larger, search the right half.

Time Complexity: $O(\log n)$

10. What is the main advantage of Divide and Conquer algorithms?

The main advantage is that they divide a complex problem into smaller and simpler subproblems, solve them independently, and combine their solutions.

Example: Merge Sort divides a large array into smaller arrays, making sorting easier and efficient.



Long Answer Questions

1. Explain the Tower of Hanoi problem in detail. Write the recursive Algorithm for solving it and describe the steps involved in moving the disks from source to destination.

The Towers of Hanoi is a mathematical puzzle used to explain the concept of recursion.

It consists of three pillars (rods):

- Source
- Auxiliary
- Destination

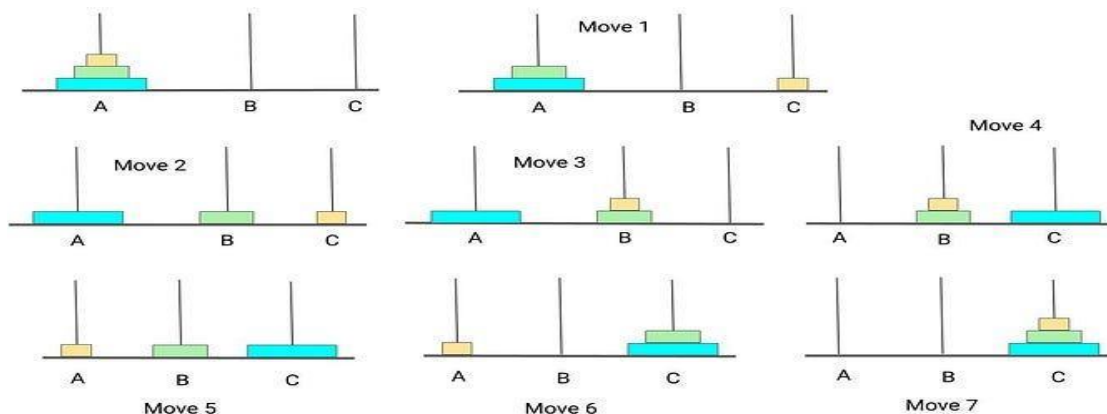
and three disks of different sizes placed on the Source rod in ascending order of size (smallest at the top, largest at the bottom).

Rules of the Problem

1. Only one disk can be moved at a time.
2. Only the top disk of any rod can be moved.
3. A larger disk cannot be placed on a smaller disk.

Objective

Move all three disks from the Source rod to the Destination rod using the Auxiliary rod, without changing the order of disks.



Steps for 3 Disks

1. Move disk 1 from Source → Destination
2. Move disk 2 from Source → Auxiliary
3. Move disk 1 from Destination → Auxiliary
4. Move disk 3 from Source → Destination
5. Move disk 1 from Auxiliary → Source



6. Move disk 2 from Auxiliary → Destination
7. Move disk 1 from Source → Destination

Total moves required = $2^3 - 1 = 7$ moves

Pseudocode for Tower of Hanoi:

Algorithm TOH(n, Source, Auxiliary, Destination)

```
{
if (n > 1 ) then
{
TOH(n - 1, Source, Destination, Auxiliary)

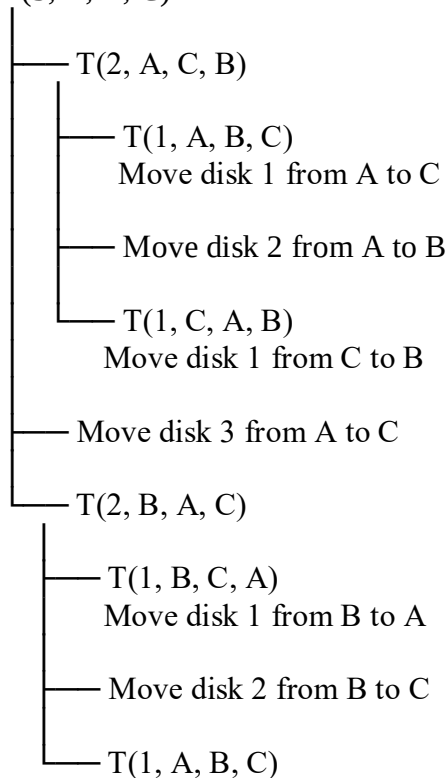
write("Move disk ", n, " from ", Source," to ", Destination)

TOH(n - 1, Auxiliary, Source, Destination)
}
}
```

Parameters

- **n** → Number of disks
- **A** → Source tower
- **B** → Destination tower
- **C** → Auxiliary (temporary) tower

T(3, A, B, C)





Move disk 1 from A to C

Solution:

$$T(n) = 2^n - 1$$

For $n = 3$:

$$T(3) = 2^3 - 1$$

$$= 8 - 1$$

$$= 7 \text{ moves}$$

Recurrence Relation :

$$T(n) = 2T(n - 1) + 1$$

Complexity

Time Complexity: $O(2^n)$

Space Complexity: $O(n)$ (due to recursion stack)

2.Explain performance analysis of an algorithm.Discuss time complexity and space complexity with suitable algorithm example.

The performance of a program is the amount of computer memory and time needed to run a program. We use two approaches to determine the performance of a program. One is analytical, and the other experimental.

In performance analysis, we use analytical methods, while in performance measurement, we conduct experiments.

Performance analysis of an algorithm can be measured in two ways

- i)By calculating its time complexity
- ii)By calculating its space complexity

Time Complexity: The time needed by an algorithm expressed as a function of the size of a problem is called the time complexity of the algorithm. The time complexity of a program is the amount of computer time it needs to run to completion.

Space Complexity:

Space complexity is the total amount of memory required by an algorithm during its execution.

It represents how much storage space an algorithm needs to solve a problem.

The memory may be required for:

- Variables
- Constants
- Arrays
- Recursion
- Dynamic memory allocation

Space Requirement Formula: $S(P) = C + S_p$

where

Fixed Part (C)

The fixed part is independent of input size.



It includes:

- Program instructions
- Constants
- Simple variables
- Fixed-size data structures

This memory remains unchanged regardless of the input.

Variable Part (SP)

The variable part depends on the size of the input.

It includes:

- Arrays
- Dynamic memory
- Recursion stack
- Variable-size data structures

As the input grows, this memory also grows.

Pseudocode

Algorithm SUM(A,n)

```
{  
sum ← 0  
  
for i ← 1 to n do  
sum ← sum + A[i]  
  
return sum  
}
```

Time Complexity Analysis

Statement	Cost	Repetition	Total
sum ← 0	1	1	1
for loop condition	1	n+1	n+1
sum ← sum + A[i]	2	n	2n
return sum	1	1	1

$$T(n) = 1 + (n + 1) + 2n + 1$$

$$T(n) = 3n + 3$$

Ignoring constants,

$$T(n) = O(n)$$



Space Complexity Analysis

Variables:

- Array A $\rightarrow$ n words
- n $\rightarrow$ 1 word
- i $\rightarrow$ 1 word
- sum $\rightarrow$ 1 word

Total space:

$$S(P) = n + 3$$

Hence,

$$S(P) = O(n)$$

3. What is Binary Search? Explain the algorithm with a suitable example and analyze its time complexity.

Binary Search : Binary Search is a searching technique used to find an element in a sorted array. It works by repeatedly dividing the search space into half.

Pseudocode for Binary Search:

Algorithm BinarySearch(A, n, key)

```
{
    low := 0;

    high := n - 1;

    while (low <= high) do
    {
        mid := (low + high) / 2;

        if (A[mid] = key) then
            return mid;

        if (key < A[mid]) then
            high := mid - 1;
        else
            low := mid + 1;
    }

    return -1;
}
```



Tracing of Binary Search

Given Array

$A = \{10, 20, 30, 40, 50, 60, 70\}$

Key = 50

Step 1:

Initialize:

low = 0

high = 6

Calculate middle element:

$mid = (low + high)/2$

$= (0 + 6)/2$

$= 3$

Therefore,

$A[3] = 40$

Compare:

$50 > 40$

Since the key is greater than the middle element, search the **right half**.

Update:

low = 4

high = 6

Step 2:

Calculate middle element:

$mid = (4 + 6)/2$

$= 5$

Therefore,

$A[5] = 60$

Compare:

$50 < 60$

Since the key is smaller than the middle element, search the **left half**.

Update:

low = 4

high = 4

Step 3:

Calculate middle element:

$mid = (4 + 4)/2$

$= 4$

Therefore,

$A[4] = 50$

Compare: $50 = 50$



The key element is found.
Return 4

Conclusion

The element 50 is found at index 4 after 3 iterations.

Time Complexity Analysis of Binary Search

Best Case

The best case occurs when the key element is found at the middle position in the very first comparison.

Example

$A = \{10, 20, 30, 40, 50, 60, 70\}$

Key = 40

Since the middle element itself is 40, only one comparison is required.

Time complexity:

Best case time complexity: $O(1)$

Average case time complexity: $O(\log n)$

Worst case time complexity: $O(\log n)$

4. Explain Strassen's Matrix Multiplication algorithm with an Example.

Strassen's Matrix Multiplication is a Divide and Conquer algorithm proposed by Volker Strassen for multiplying two matrices faster than the conventional method.

The ordinary matrix multiplication of two $n \times n$ matrices require:

$$O(n^3)$$

time complexity.

Strassen's algorithm reduces this to:

$$\begin{aligned} O(n^{\log_2 7}) \\ O(n^{2.807}) \end{aligned}$$

which is faster for large matrices.

Divide and Conquer Strategy

Given two matrices A and B of order $n \times n$, divide each matrix into four submatrices of size $\frac{n}{2} \times \frac{n}{2}$.

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}$$
$$B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

The resultant matrix is

$$C = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}$$



Conventional Method

Normally,

$$\begin{aligned}C_{11} &= A_{11}B_{11} + A_{12}B_{21} \\C_{12} &= A_{11}B_{12} + A_{12}B_{22} \\C_{21} &= A_{21}B_{11} + A_{22}B_{21} \\C_{22} &= A_{21}B_{12} + A_{22}B_{22}\end{aligned}$$

This requires 8 multiplications.

Strassen's Improvement

Instead of 8 multiplications, Strassen computes only 7 multiplications.

Step 1: Compute Seven Products

$$\begin{aligned}P &= (A_{11} + A_{22})(B_{11} + B_{22}) \\Q &= (A_{21} + A_{22})B_{11} \\R &= A_{11}(B_{12} - B_{22}) \\S &= A_{22}(B_{21} - B_{11}) \\T &= (A_{11} + A_{12})B_{22} \\U &= (A_{21} - A_{11})(B_{11} + B_{12}) \\V &= (A_{12} - A_{22})(B_{21} + B_{22})\end{aligned}$$

Step 2: Compute Result Matrix

$$\begin{aligned}C_{11} &= P + S - T + V \\C_{12} &= R + T \\C_{21} &= Q + S \\C_{22} &= P + R - Q + U\end{aligned}$$

Example

Multiply

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

and

$$B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

Step 1: Identify Elements

$$\begin{aligned}A_{11} &= 1, A_{12} = 2, A_{21} = 3, A_{22} = 4 \\B_{11} &= 5, B_{12} = 6, B_{21} = 7, B_{22} = 8\end{aligned}$$

Step 2: Calculate the Seven Products

$$\begin{aligned}P &= (1 + 4)(5 + 8) = 5 \times 13 = 65 \\Q &= (3 + 4)(5) = 7 \times 5 = 35 \\R &= (1)(6 - 8) = 1 \times (-2) = -2 \\S &= (4)(7 - 5) = 4 \times 2 = 8 \\T &= (1 + 2)(8) = 3 \times 8 = 24 \\U &= (3 - 1)(5 + 6) = 2 \times 11 = 22 \\V &= (2 - 4)(7 + 8) = (-2) \times 15 = -30\end{aligned}$$



Step 3: Compute Result Matrix

$$C_{11} = 65 + 8 - 24 - 30 = 19$$

$$C_{12} = -2 + 24 = 22$$

$$C_{21} = 35 + 8 = 43$$

$$C_{22} = 65 - 35 - 2 + 22 = 50$$

Final Result

$$C = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$$

Recurrence Relation

$$T(n) = 7T\left(\frac{n}{2}\right) + O(n^2)$$

5. Explain the Divide and Conquer strategy with an algorithm and discuss its applications.

- Divide and Conquer is an algorithm design technique in which a problem is repeatedly divided into smaller subproblems of the same type, each subproblem is solved independently, and their solutions are combined to obtain the final solution.
- This technique is widely used to design efficient algorithms for sorting, searching, matrix multiplication, and many other problems.

Basic Steps

1. Divide

Break the given problem into smaller subproblems.

2. Conquer

Solve each subproblem recursively. If the subproblem is small enough, solve it directly.

3. Combine

Merge the solutions of the subproblems to obtain the solution to the original problem.

Control Abstraction of Divide and Conquer

The control abstraction for divide and conquer technique is DANDC(P), where P is the problem to be solved.

DANDC (P)

```
{  
if SMALL (P) then return S (p);else  
{  
divide p into smaller instances p1, p2, .... Pk, k 1;apply DANDC to each of these sub problems;
```



```
return (COMBINE (DANDC (p1) , DANDC (p2),..., DANDC (pk));  
}  
}
```

SMALL (P) is a Boolean-valued function which determines whether the input size is small enough so that the answer can be computed without splitting. If this is so, function 'S' is invoked; otherwise, the problem 'p' is split into smaller subproblems. These subproblems p1, p2, . . . , pk are solved by recursive application of DANDC.

Recurrence Relation

The running time of Divide and Conquer algorithms is generally expressed as:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

Where:

- **a** = Number of subproblems
- **n/b** = Size of each subproblem
- **f(n)** = Time required to divide and combine

Applications of Divide and Conquer:

1. Binary Search
2. Quick sort
3. Merge sort
4. Strassen's Matrix Multiplication

6. Explain Merge sort Algorithm with an example and analyze its time complexity.

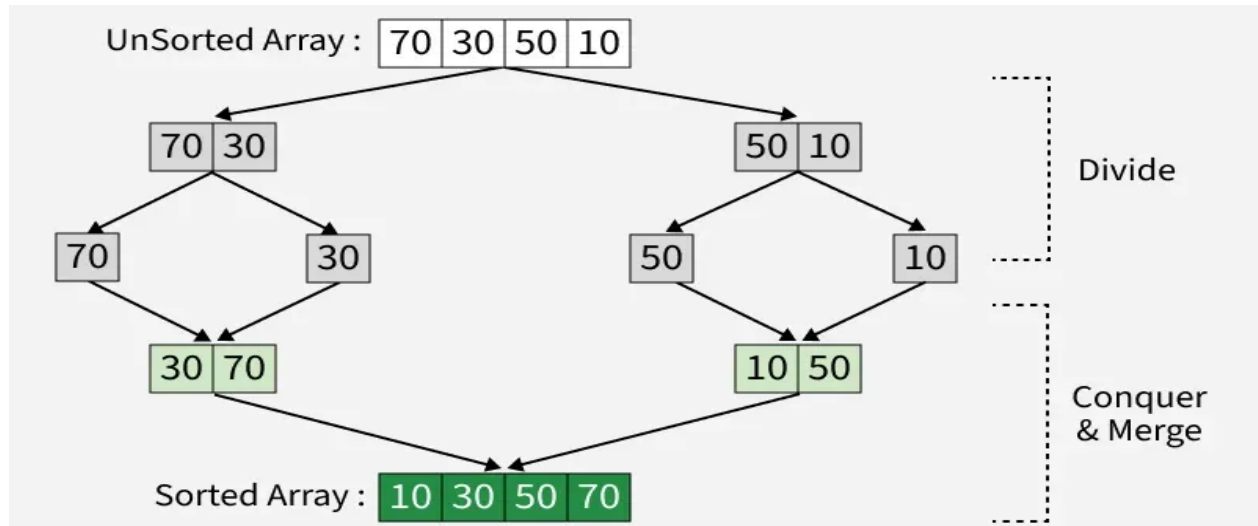
Merge Sort :

Merge Sort is a divide-and-conquer sorting algorithm that divides the array into smaller parts, sorts them, and then merges them back together in sorted order.

The array is split into smaller and smaller pieces until it is merged back together. And as the merging happens, values from each sub-array are compared so that the lowest value comes first.

Here's a step-by-step explanation of how merge sort works:

1. **Divide:** Divide the list or array recursively into two halves until it can no more be divided.
2. **Conquer:** Each subarray is sorted individually using the merge sort algorithm.
3. **Merge:** The sorted subarrays are merged back together in sorted order. The process continues until all elements from both subarrays have been merged.



Here, a problem is divided into multiple sub-problems. Each sub-problem is solved individually. Finally, sub-problems are combined to form the final solution.

Merge Sort Algorithm

Algorithm MergeSort(A, low, high)

```
{
  if (low < high) then
  {
    mid := (low + high)/2;

    MergeSort(A, low, mid);

    MergeSort(A, mid+1, high);

    Merge(A, low, mid, high);
  }
}
```



Merge Procedure

Algorithm MERGE(A, low, mid, high)

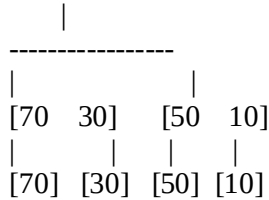
1. Compare the elements of the left and right subarrays.
2. Copy the smaller element into a temporary array.
3. Repeat until one subarray becomes empty.
4. Copy the remaining elements of the other subarray.
5. Copy the temporary array back to A[low...high].

Tracing for the Given Diagram

Input: A = [70, 30, 50, 10]

Divide Phase

[70 30 50 10]



Merge Phase

Merge [70] and [30]

→ [30 70]

Merge [50] and [10]

→ [10 50]

Merge [30 70] and [10 50]

Compare 30 and 10 → 10

Compare 30 and 50 → 30

Compare 70 and 50 → 50

Remaining → 70

Final Sorted Array:

[10 30 50 70]



Time Complexity Analysis of Merge Sort:

Recurrence Relation

$$T(n) = 2T\left(\frac{n}{2}\right) + n$$

Case	Time Complexity
Best Case	$O(n \log n)$
Average Case	$O(n \log n)$
Worst Case	$O(n \log n)$

7. Define time complexity. Explain asymptotic notations such as Big-O, Omega (Ω), and Theta (Θ)

Time Complexity

Time complexity is the amount of computer time required by an algorithm to execute and produce the desired output.

Asymptotic Notations

Asymptotic notation tells us how an algorithm behaves for large input sizes without considering machine speed, programming language, or constant factors.

Following asymptotic notations are used to calculate the running time complexity of an algorithm.

- **O** – Big Oh Notation
- **Ω** – Big omega Notation
- **Θ** – Big theta Notation
- **o** – Little Oh Notation
- **ω** – Little omega Notation

Big Oh, O: Asymptotic Upper Bound

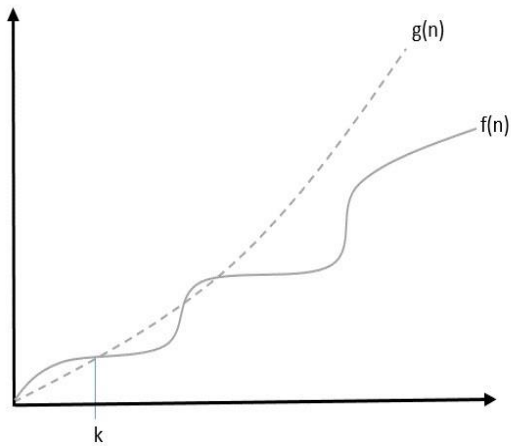
Big-O notation represents the upper bound of an algorithm's running time. It specifies the maximum amount of time an algorithm may take for a given input size.

It is generally used to represent the **worst-case complexity**.

A function $f(n)$ can be represented is the order of $g(n)$ that is $O(g(n))$, if there exists a value of positive integer n as n_0 and a positive constant c such that

$f(n) \leq c \cdot g(n)$ for $n > n_0$ in all case

Hence, function $g(n)$ is an upper bound for function $f(n)$, as $g(n)$ grows faster than $f(n)$.



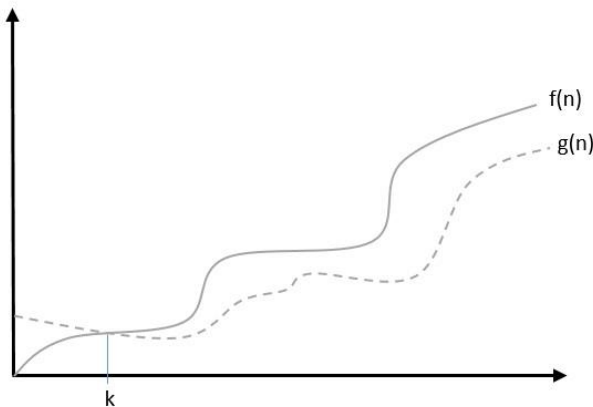
Big Omega, Ω : Asymptotic Lower Bound

Omega notation represents the lower bound of an algorithm's running time. It specifies the minimum amount of time required by an algorithm. It is generally used to represent the **best-case complexity**.

We say that $f(n) = \Omega(g(n))$

when there exists constant c that $f(n) \geq c \cdot g(n)$

for all sufficiently large value of n . Here n is a positive integer. It means function g is a lower bound for function f ; after a certain value of n , f will never go below g .



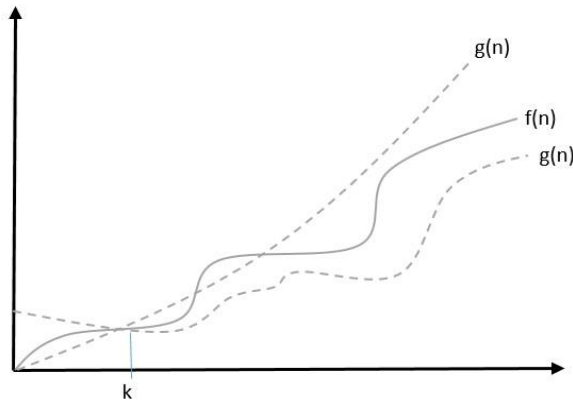
Theta, Θ : Asymptotic Tight Bound

Theta notation (Θ) is used to represent the exact growth rate of an algorithm by providing both upper and lower bounds. It represents the tight bound of an algorithm's running time. It specifies both the upper and lower bounds and gives the exact growth rate.

We say that $f(n) = \Theta(g(n))$ when there exist constants c_1 and c_2 that $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$

for all sufficiently large value of n . Here n is a positive integer.

This means function g is a tight bound for function f .



8.What is Algorithm Analysis? Explain Algorithm Specification and describe the different methods of representing an algorithm. Discuss the various Algorithm Design Strategies with suitable examples.

Introduction to Algorithm:

Algorithm is a step-by-step procedure, which defines a set of instructions to be executed in a certain order to get the desired output. Algorithms are generally created independent of underlying languages, i.e. an algorithm can be implemented in more than one programming language.

Characteristics of an Algorithm

An algorithm should have the following characteristics –

- **Unambiguous** – Algorithm should be clear and unambiguous. Each of its steps (or phases), and their inputs/outputs should be clear and must lead to only one meaning.
- **Input** – An algorithm should have 0 or more well-defined inputs.
- **Output** – An algorithm should have 1 or more well-defined outputs, and should match the desired output.
- **Finiteness** – Algorithms must terminate after a finite number of steps.
- **Feasibility** – Should be feasible with the available resources.
- **Independent** – An algorithm should have step-by-step directions, which should be independent of any programming code.

How to express an Algorithm?



- Natural Language: Written in plain English. Easy to describe, but can be unclear for complex problems

Flowchart: Graphical representation of steps. Easier to visualize than natural language.

- Pseudocode: Text-based, code-like instructions without language syntax.
 - Best way to express an algorithm.
 - Understandable even to beginners with basic knowledge.

Rules for Writing Pseudocode

1. Comments: Use // to explain the purpose of statements.
2. Blocks and Statements: Group statements using { } and separate statements using ;.
3. Identifiers: Identifiers should begin with a letter. Data types need not be explicitly declared.
4. Assignment: Use := for assigning values.
Example: `sum := a + b;`
5. Operators: Use logical operators (and, or, not) and relational operators (<, >, =, <=, >=, <>).
6. Arrays: Use square brackets [] to access array elements. Indexing starts from 0.
Example: `A[i]`
7. Loops: Use for, while, and repeat-until for repetition. break terminates the loop.
8. Conditional Statements: Use if, if-else, and case for decision making.
9. Input/Output: Use read() for input and write() for output.
Example: `read(n);` and `write(result);`
10. Algorithm Structure: An algorithm consists of a heading and a body containing the required steps.

General Form:

```
Algorithm Name(parameter_list)
{
    statements;
}
```

The body contains one or more statements enclosed within braces. An algorithm may or may not return a value. Simple variables are passed by value, whereas arrays and records are passed by reference.

Example 1: Addition of Two Numbers

Problem

Write an algorithm to find the sum of two numbers.

Pseudocode



```
Algorithm AddNumbers(a, b)
{
    // Calculate the sum of two numbers
    sum := a + b;
    write(sum);
}
```

9. Compare Quick Sort and Merge Sort based on their working principle, And mention their time complexity, space complexities.

Both Quick Sort and Merge Sort are based on the Divide and Conquer technique.

Comparison

Basis	Quick Sort	Merge Sort
Working Principle	Selects a pivot and partitions the array into two parts.	Divides the array into two halves and merges the sorted halves.
Divide	Based on the position of the pivot.	Divides the array into two halves.
Conquer	Recursively sorts the left and right partitions.	Recursively sorts the two halves.
Combine	No separate combine step is required.	Sorted halves are combined using merge operation.
Best Case	$O(n \log n)$	$O(n \log n)$
Average Case	$O(n \log n)$	$O(n \log n)$
Worst Case	$O(n^2)$	$O(n \log n)$
Space Complexity	$O(\log n)$ average; $O(n)$ worst case	$O(n)$
Stable	No	Yes

Working Principle

Quick Sort:

Array $\rightarrow$ Select Pivot $\rightarrow$ Partition $\rightarrow$ Sort Left & Right Parts

Merge Sort:

Array $\rightarrow$ Divide $\rightarrow$ Sort Subarrays $\rightarrow$ Merge

Time and Space Complexities

Quick Sort:

- Best Case = $O(n \log n)$
- Average Case = $O(n \log n)$
- Worst Case = $O(n^2)$
- Space Complexity = $O(\log n)$ average, $O(n)$ worst case

Merge Sort:



- Best Case = $O(n \log n)$
- Average Case = $O(n \log n)$
- Worst Case = $O(n \log n)$
- Space Complexity = $O(n)$

10.Explain the Divide and Conquer strategy in detail. Discuss its three fundamental steps with a suitable example.

Divide and Conquer Technique

Divide and Conquer is an algorithm design technique in which a problem is repeatedly divided into smaller subproblems of the same type, each subproblem is solved independently, and their solutions are combined to obtain the final solution.

This technique is widely used to design efficient algorithms for sorting, searching, matrix multiplication, and many other problems.

Basic Steps

1. Divide

Break the given problem into smaller subproblems.

2. Conquer

Solve each subproblem recursively. If the subproblem is small enough, solve it directly.

3. Combine

Merge the solutions of the subproblems to obtain the solution to the original problem.

Control Abstraction of Divide and Conquer

A control abstraction is a procedure whose flow of control is clear but whose primary operations are specified by other procedures whose precise meanings are left undefined. The control abstraction for divide and conquer technique is DANDC(P), where P is the problem to be solved.

ALGORITHM DANDC(P)

```
{
  if SMALL(P) then
    return S(P);
  else
  {
    Divide P into smaller subproblems
    P1, P2, ..., Pk;
    Apply DANDC to each subproblem;
    return COMBINE(
      DANDC(P1),
```



```
DANDC(P2),  
...  
DANDC(Pk)  
);  
}  
}
```

If the sizes of the two sub problems are approximately equal then the computing time of DANDC
Where, $T(n)$ is the time for DANDC on 'n' inputs
 $g(n)$ is the time to complete the answer directly for small inputs, and $f(n)$ is the time for Divide and Combine.



MODULE – II

Short Answer Questions

1. What is Hamiltonian cycle?

A Hamiltonian cycle is a cycle in a graph that visits every vertex exactly once and returns to the starting vertex.

Example:

In a graph with vertices A, B, C, and D, the path $A \rightarrow B \rightarrow C \rightarrow D \rightarrow A$ is a Hamiltonian cycle if each vertex is visited exactly once before returning to A.

2. Write an Algorithm for Simple Union Operation?

The **Union** operation combines two disjoint sets into a single set.

Algorithm: UNION(i, j)

```
UNION(i, j)
{
    parent[j] ← i;
}
```

Explanation:

If i and j are the roots (representatives) of two different sets, make i the parent of j, thereby merging the two sets into one.

Example:

If Set 1 = {1, 2} with root 1 and Set 2 = {3, 4} with root 3,

UNION(1, 3)

Then 1 becomes the parent of 3, and the sets are merged.

3. What is the state-space tree in backtracking?

A state-space tree is a graphical representation of all possible solutions explored during backtracking.

4. Define the Find operation in a disjoint-set data structure.

The Find operation determines the representative (root) of the set containing a given element.

Given the element i, find the set containing i.

Example:

S1 = {1,7,8,9} S2 = {2,5,10} S3 = {3,4,6}

Then,

Find(4) = S3 Find(5) = S2 Find(7) = S1



5. How is a Heap Implemented?

A heap is implemented using an array representation of a complete binary tree. The parent and child nodes are accessed using index calculations, making insertion and deletion efficient.

6. What is the Sum of Subsets Problem?

The Sum of Subsets problem is to find all subsets of a given set whose elements add up to a specified target sum using backtracking.

7. Define Max Heap and Min Heap.

- Max Heap: A complete binary tree in which the value of every parent node is greater than or equal to its children. The root contains the maximum element.
- Min Heap: A complete binary tree in which the value of every parent node is less than or equal to its children. The root contains the minimum element.

8. Write the general algorithm for Backtracking.

Backtracking is a systematic method for solving problems by constructing a solution step-by-step and abandoning a solution when it cannot lead to a valid answer.

```
Backtrack(k)
{
    if solution found
        print solution
    else
    {
        for each candidate x
        {
            if x is promising
            {
                include x
                Backtrack(k+1)
                remove x
            }
        }
    }
}
```

9. What is the Sum of Subsets Problem?

The Sum of Subsets Problem is the problem of finding one or more subsets of a given set of positive integers whose sum is equal to a specified target value.

Example:

Set = {5, 10, 12, 13, 15}, Target = 25



A solution is {10, 15} because $10 + 15 = 25$.

10. What is the Graph coloring?

Graph Coloring is the process of assigning colors to the vertices of a graph such that no two adjacent vertices have the same color. The objective is usually to use the minimum number of colors.

Long Answer Questions

1.Explain Disjoint Set Operations. Describe Union and Find algorithms with examples.

Disjoint Sets:

The disjoint sets are those that do not have any common element.

For example $S_1 = \{1, 7, 8, 9\}$ and $S_2 = \{2, 5, 10\}$, then we can say that S_1 and S_2 are two disjoint sets.

Disjoint Set Operations:

The disjoint set operations are

1. Union
2. Find

Disjoint set Union:

If S_i and S_j are two disjoint sets, then their union $S_i \cup S_j$ consists of all the elements x such that x is in S_i or S_j .

Example:

$S_1 = \{1, 7, 8, 9\}$ $S_2 = \{2, 5, 10\}$

$S_1 \cup S_2 = \{1, 2, 5, 7, 8, 9, 10\}$

FIND: The **Simple Find** operation finds the root node (representative element) of the set containing a given element.

Given the element i , find the set containing i .

Example:

$S_1 = \{1, 7, 8, 9\}$ $S_2 = \{2, 5, 10\}$ $S_3 = \{3, 4, 6\}$

Then,

$\text{Find}(4) = S_3$ $\text{Find}(5) = S_2$ $\text{Find}(7) = S_1$

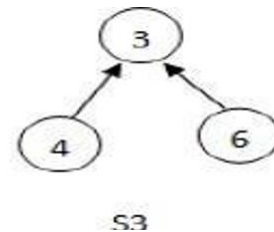
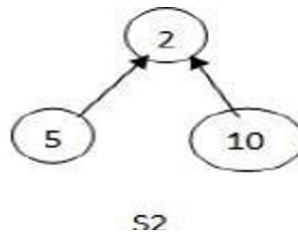
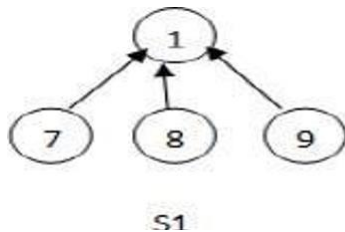
Set Representation:

The set will be represented as the tree structure where all children will store the address of parent / root node. The root node will store null at the place of parent address. In the given set of elements any element can be selected as the root node, generally we select the first node as the root node.

Example:

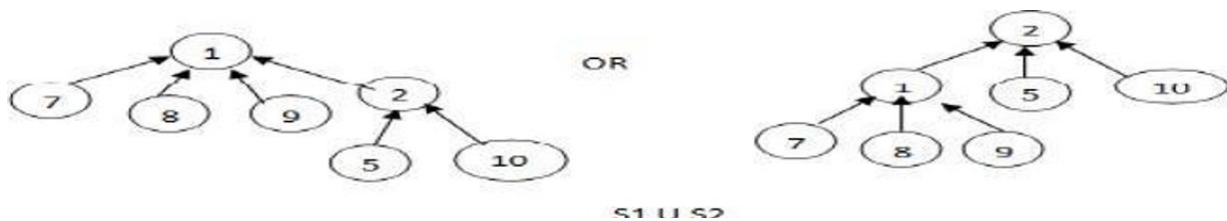
$S_1 = \{1, 7, 8, 9\}$ $S_2 = \{2, 5, 10\}$ $S_3 = \{3, 4, 6\}$

Then these sets can be represented as



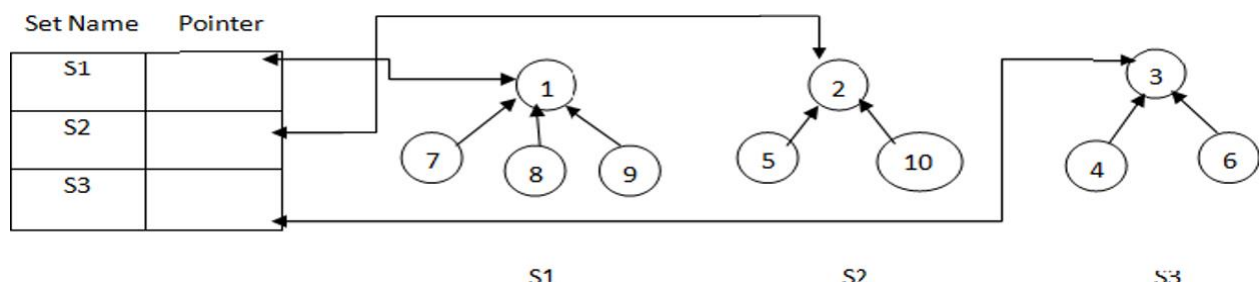
Disjoint Union:

To perform disjoint set union between two sets S_i and S_j can take any one root and make it sub-tree of the other. Consider the above example sets S1 and S2 then the union of S1 and S2 can be represented as any one of the following.



Find:

To perform find operation, along with the tree structure we need to maintain the name of each set. So, we require one more data structure to store the set names. The data structure contains two fields. One is the set name and the other one is the pointer to root.





UNION AND FIND ALGORITHMS:

Rules for Simple Union

1. Consider all elements as individual sets.
2. Initialize the parent array with -1.
3. Select two root nodes.
4. Make one root the parent of the other.
5. Update the parent array after every union.

Algorithm

SimpleUnion(i, j)

```
{  
    P[j] = i;  
}
```

where

- i = root of first set
- j = root of second set

After union, i becomes the parent of j.

Example

Consider the elements

1 2 3 4 5 6

Step 1: Initial State

P[1]=-1

P[2]=-1

P[3]=-1

P[4]=-1

P[5]=-1

P[6]=-1

Tree Representation

1 2 3 4 5 6

Each element is a separate set.

Step 2: Union(1,2)

P[2] = 1

Tree

1

|

2

Parent Array

i	1	2	3	4	5	6
---	---	---	---	---	---	---

P[i]	-1	1	-1	-1	-1	-1
------	----	---	----	----	----	----

Step 3: Union(3,4)

P[4] = 3



Tree

3

|

4

Parent Array

i 1 2 3 4 5 6

P[i] -1 1 -1 3 -1 -1

Step 4: Union(5,6)

P[6] = 5

Tree

5

|

6

Parent Array

i 1 2 3 4 5 6

P[i] -1 1 -1 3 -1 5

Step 5: Union(1,3)

P[3] = 1

Tree

1

/ \

2 3

|

4

Parent Array

i 1 2 3 4 5 6

P[i] -1 1 1 3 -1 5

Step 6: Union(1,5)

P[5] = 1

Final Tree

1

/ | \

2 3 5

| |

4 6

Final Parent Array

i 1 2 3 4 5 6

P[i] -1 1 1 3 1 5



Simple Find

Definition

The **Simple Find** operation finds the root node (representative element) of the set containing a given element.

Algorithm

```
Find(i)
{
    while(P[i] > 0)
    {
        i = P[i];
    }

    return i;
}
```

Rules for Simple Find

1. Start from the given node.
2. Follow parent links.
3. Continue until a node with parent -1 is reached.
4. That node is the root.
5. Return the root.

Example

Find(4)

Step 1

i = 4

P[4] = 3

Move to parent.

i = 3

Step 2

P[3] = 1

Move to parent.

i = 1

Step 3

P[1] = -1

Root found.

Return 1



Tracing Table for Find(4)

Step	i	P[i]
1	4	3
2	3	1
3	1	-1

Therefore,

Find(4) = 1

Time Complexity

Operation Complexity

Simple Union $O(1)$

Simple Find $O(n)$ (Worst Case)

2. Explain the N-Queens Problem using Backtracking. Illustrate the solution for a 4-Queens problem.

N-Queens Problem: A classic combinational problem is to place n queens on a $n \times n$ chess board so that no two attack, i.e. no two queens are on the same row, column or diagonal. If we take $n=4$ then the problem is called 4 queens problem. If we take $n=8$ then the problem is called as 8 queens problem.

4-Queens problem: Consider a 4×4 chessboard. Let there are 4 queens. The objective is place there 4 queens on 4×4 chessboard in such a way that no two queens should be placed in the same row, same column or diagonal position. The explicit constraints are 4 queens are to be placed on 4×4 chessboards in 44 ways. The implicit constraints are no two queens are in the same row column or diagonal.

Let $\{x_1, x_2, x_3, x_4\}$ be the solution vector where x_i column on which the queen i is placed.

First queen is placed in first row and first column.

Q1			

The second queen should not be in first row and second column. It should be placed in second row and in second, third or fourth column. If we place in second column, both will be in same diagonal, so place it in third column.

Q1			
X	X		



Q1			
x	x	Q2	

Q1			
x	x	Q2	
x	x	x	x

We are unable to place queen 3 in third row, so go back to queen 2 and place it somewhere else

Q1			
			Q2
	Q3		

Q1			
			Q2
	Q3		
x	x	x	x

Now the fourth queen should be placed in 4th row and 3rd column but there will be a diagonal attack from queen 3. So go back, remove queen 3 and place it in the next column. But it is not possible, so move back to queen 2 and remove it to next column but it is not possible. So go back to queen 1 and move it to next column.

	Q1		

	Q1		
			Q2

	Q1		
			Q2
Q3			

	Q1		
			Q2
Q3			
	Q4		

Fig: Example of Backtrack solution to the 4-queens problem

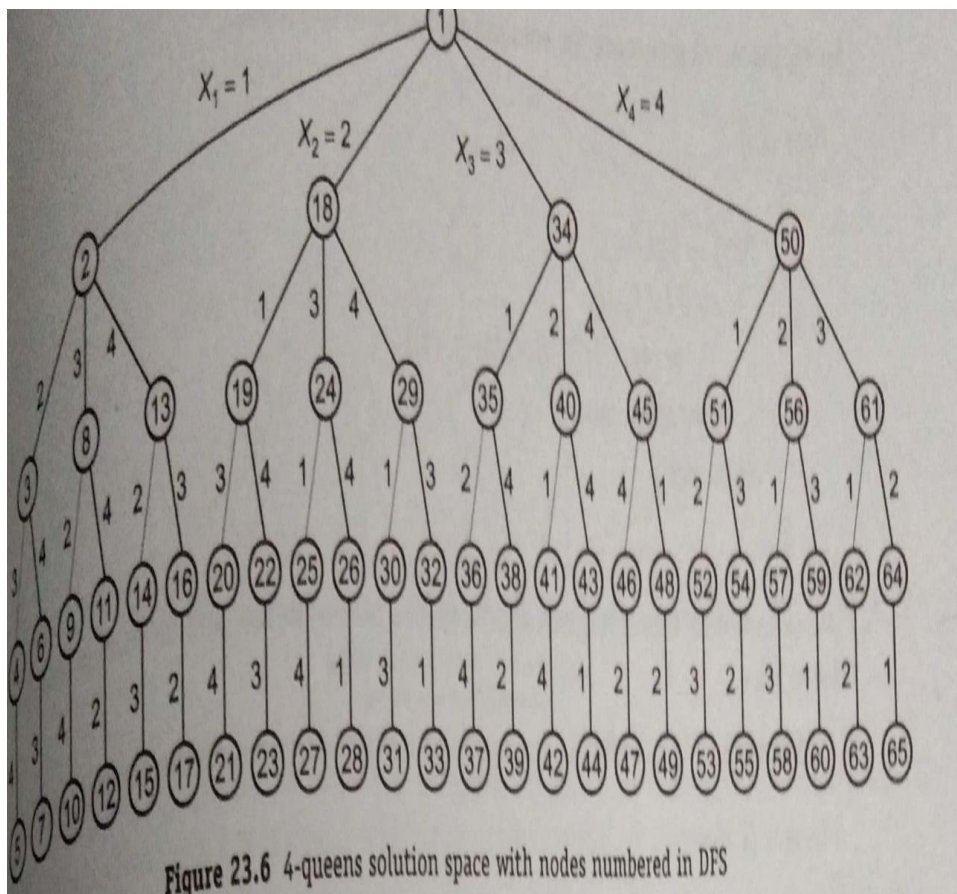
Hence the solution of to 4-queens's problem is $x_1=2, x_2=4, x_3=1, x_4=3$, i.e first queen is placed in 2nd column, second queen is placed in 4th column and third queen is placed in first column and fourth queen is placed in third column.



Algorithm NQueens

Algorithm NQueens(row)

1. If (row > N) then
2. Print(solution)
3. return
4. End If
5. For col = 1 to N do
6. If isSafe(row, col) then
7. PlaceQueen(row, col)
8. NQueens(row + 1)
9. RemoveQueen(row, col) // Backtracking
10. End If
11. End For





3. Describe the general method of Backtracking with an algorithm and write its applications.

Definition: The backtracking algorithm is a problem-solving approach that tries out all the possible solutions and chooses the best or desired ones. Generally, it is used to solve problems that have multiple solutions.

Terminology Used in Backtracking

Root Node

Represents the initial state of the problem.

Live Node

A node that has been generated but not yet explored.

E-node (Expansion Node)

The live node currently being expanded.

Dead Node

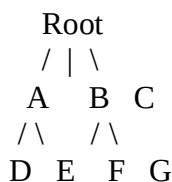
A node that cannot lead to a solution and is discarded.

Answer Node

A node representing a complete valid solution.

State Space Tree

A **State Space Tree** is a graphical representation of all possible states generated during the search.



- Root → Initial state
- Internal nodes → Partial solutions
- Leaf nodes → Complete solutions
- Invalid branches are pruned

Backtracking Algorithm

Backtrack(k)

```
{
  if solution found
    print solution
  else
    {
      for each candidate x
    {
      if x is promising
      {
        include x
        Backtrack(k+1)
        remove x
      }
    }
  }
}
```



```
}  
}  
}  
}
```

Steps in Backtracking

1. Start from the root node.
2. Generate a candidate solution.
3. Check whether it is promising.
4. If promising, continue searching.
5. If not promising, backtrack.
6. Repeat until all solutions are explored.

Applications of Backtracking

1. N-Queens Problem
2. Graph Coloring
3. Hamiltonian Cycle
4. Sum of Subsets

4. Generate the state space tree and find the solutions for the subset sum for $N=4$, $M=50$, $W=\{10,20,30,40\}$ using backtracking approach.

Definition

Given a set of positive integers $S = \{s_1, s_2, \dots, s_n\}$ and a target value d , the objective is to find all subsets of S whose sum is exactly equal to d . The problem is commonly solved using the Backtracking technique, where non-promising subsets are discarded to reduce the search space.

For the Sum of Subsets problem,

- Set: $S = \{10, 20, 30, 40\}$
- Target Sum = 50
- Let
 - x_1 represent element 10
 - x_2 represent element 20
 - x_3 represent element 30
 - x_4 represent element 40
- 1 = Include the element
- 0 = Exclude the element



Algorithm for Sum of Subset

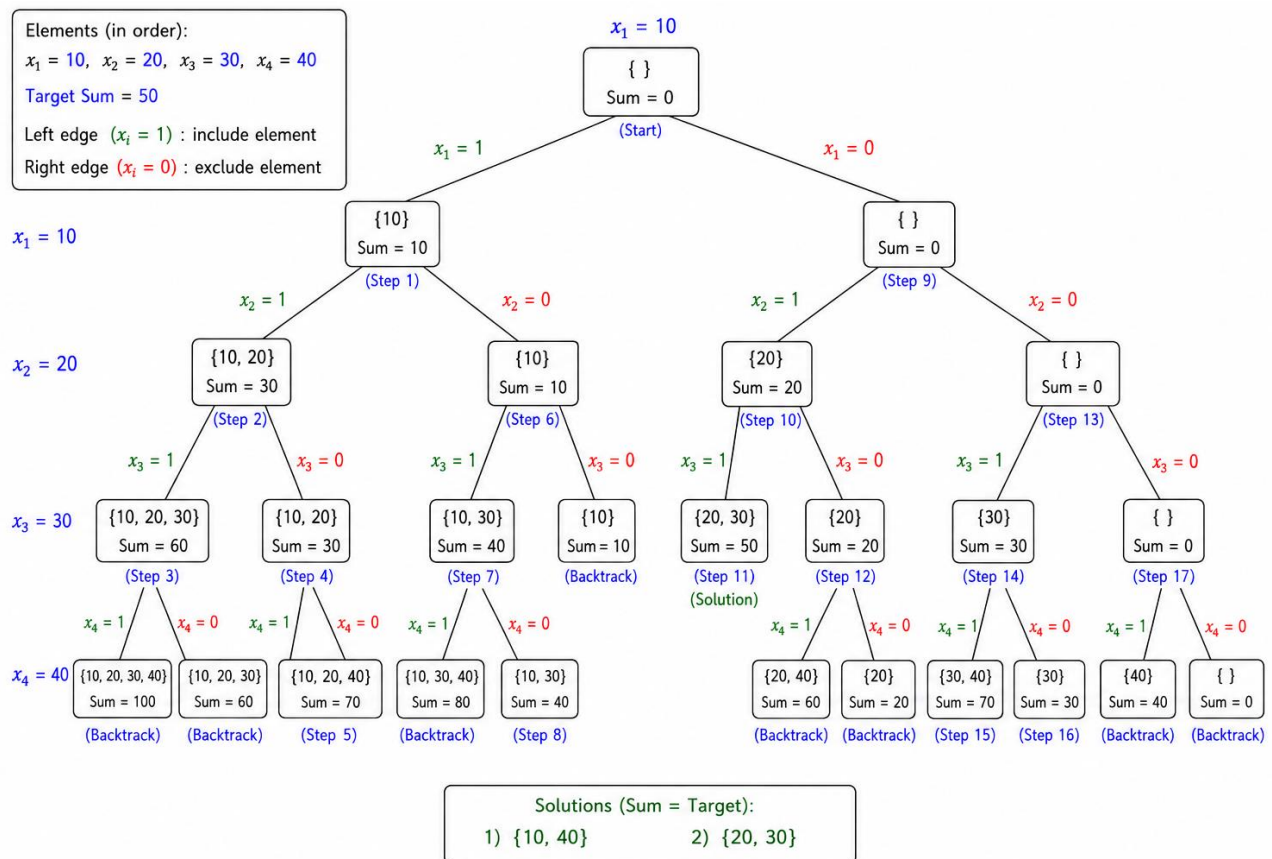
1. If $\text{sum} = d$ then
 Print the subset
 Return
2. If $\text{sum} > d$ or $i = n$ then
 Return
3. Include $S[i]$ in the subset
 SumOfSubsets($i + 1, \text{sum} + S[i]$)
4. Exclude $S[i]$ from the subset
 SumOfSubsets($i + 1, \text{sum}$)

Explanation

- $i \rightarrow$ Current element index.
- **sum** $\rightarrow$ Sum of elements selected so far.
- **d** $\rightarrow$ Target sum.
- **S[i]** $\rightarrow$ Current element being considered.

Search Table

Step	Current Subset	Sum	Action
1	{}	0	Start
2	{10}	10	Add next
3	{10,20}	30	Add next
4	{10,20,30}	60	Backtrack
5	{10,20,40}	70	Backtrack
6	{10,30}	40	Add next
7	{10,30,40}	80	Backtrack
8	{10,40}	50	Solution
9	{20}	20	Continue
10	{20,30}	50	Solution



5. Explain the Graph Coloring problem and solve it using the Backtracking technique.

Definition:

Let G be a graph and m be a given positive integer. We want to discover whether the nodes of G can be colored so that no two adjacent nodes have the same color, using only m colors. This is called the m -colorability decision problem.

Notes:

- If d is the degree of the graph, it can be colored with $d+1$ colors.
- The smallest value of m required is called the chromatic number.

General Backtracking Algorithm

mColoring(k)

while (true)

 NextValue(k)

 if $x[k] == 0$
 return



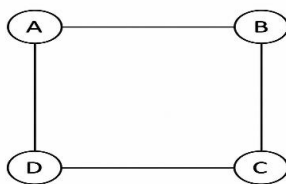
```
if k == n
    print(x[1...n])

else
    mColoring(k+1)
```

Algorithm NextValue(k)

1. Repeat
2. Assign next color to vertex k.
3. If no color is available, return.
4. Compare the color of vertex k with all adjacent vertices.
5. If any adjacent vertex has the same color, try the next color.
6. Otherwise, accept the color and return.
7. Repeat until a valid color is obtained.

Example Graph



Vertices: A, B, C, D

Edges:

- A — B
- B — C
- C — D
- D — A

Adjacency of Vertices

- A is adjacent to B and D
- B is adjacent to A and C
- C is adjacent to B and D
- D is adjacent to A and C

Adjacency Matrix

	V1	V2	V3	V4
V1	0	1	0	1
V2	1	0	1	0
V3	0	1	0	1
V4	1	0	1	0



Coloring Process

Initially

$x = [0, 0, 0, 0]$

where

- $0 \rightarrow$ No color assigned
- $1 \rightarrow$ Red
- $2 \rightarrow$ Green
- $3 \rightarrow$ Blue

Step 1

Assign **V1 (A) = Color 1 (Red)**

$x = [1, 0, 0, 0]$

Step 2

Assign V2 (B)

V2 is adjacent to V1.

- Color 1 (Red) X (Same as V1)
- Color 2 (Green) ✓
- Color 3 (Blue) ✓

Choose Color 2 (Green).

$x = [1, 2, 0, 0]$

Step 3

Assign V3 (C)

V3 is adjacent only to V2.

- Color 1 (Red) ✓
- Color 2 (Green) X (Same as V2)
- Color 3 (Blue) ✓

Choose Color 1 (Red) (first valid color).

$x = [1, 2, 1, 0]$

Step 4

Assign V4 (D)

V4 is adjacent to V1 and V3.

- Color 1 (Red) X (Same as V1 and V3)
- Color 2 (Green) ✓
- Color 3 (Blue) ✓

Choose Color 2 (Green).

$x = [1, 2, 1, 2]$

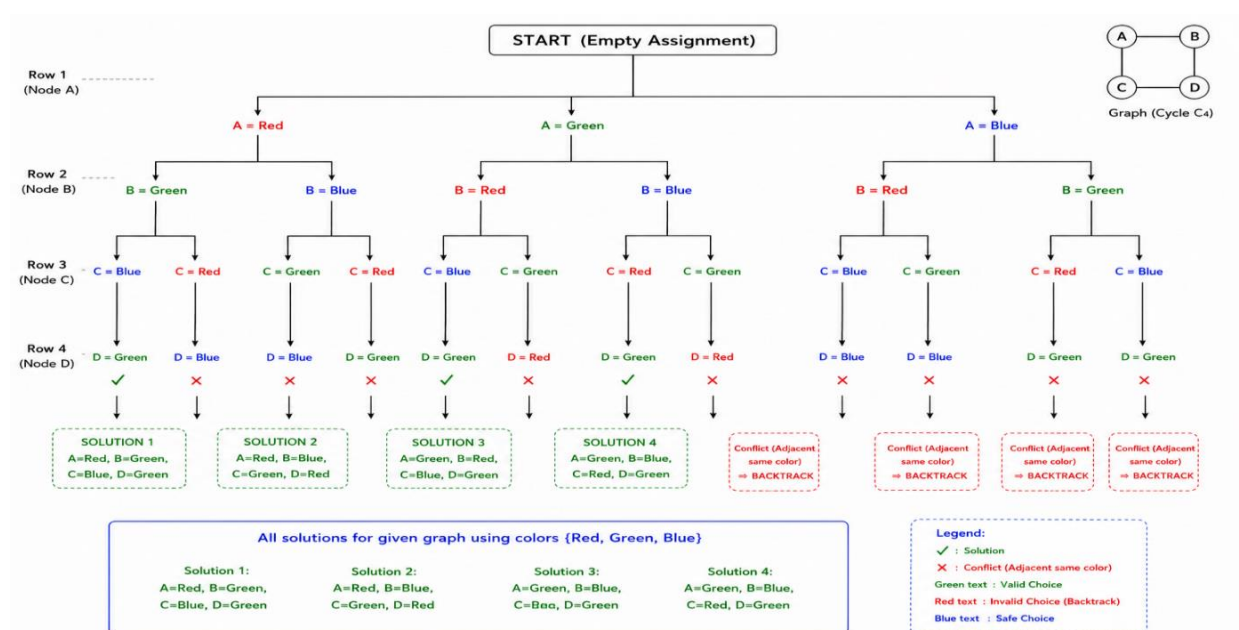


Final Coloring

Vertex	Color Number	Color Name
V1 (A)	1	Red
V2 (B)	2	Green
V3 (C)	1	Red
V4 (D)	2	Green

Final Color Assignment

V1 (A) → Color 1 (Red)
V2 (B) → Color 2 (Green)
V3 (C) → Color 1 (Red)
V4 (D) → Color 2 (Green)



Time Complexity

- **Worst Case:** $O(m^n)O(mn)$
 - nn = number of vertices
 - mm = number of colors

Space Complexity

- $O(n)$ for the recursion stack and color array.



6. What is a Hamiltonian Cycle? Explain the Backtracking solution for finding Hamiltonian Cycles in a graph.

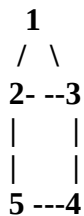
A **Hamiltonian Cycle** is a cycle in a graph that **visits every vertex exactly once and returns to the starting vertex**.

- Each vertex is visited only once.
- The first and last vertices are the same.
- Not every graph contains a Hamiltonian cycle.

Problem Statement

Given an undirected graph $G(V, E)$, determine whether there exists a Hamiltonian cycle.

Example Graph



One Hamiltonian Cycle is:

$1 \rightarrow 2 \rightarrow 5 \rightarrow 4 \rightarrow 3 \rightarrow 1$

Idea of Backtracking

- Start from the first vertex.
- Add one vertex at a time to the path.
- Before adding a vertex, check whether it is valid.
- If a wrong choice is made, **go back (backtrack)** and try another vertex.
- Continue until all vertices are included in the cycle.

Conditions to Add a Vertex

A vertex can be added only if:

1. It is connected to the previous vertex.
2. It has not been used before.
3. If it is the last vertex, it must connect back to the starting vertex.

ALGORITHM:

Hamiltonian(k)

If all vertices are included

Print the cycle

Else

Try every vertex

If the vertex is valid

Add it to the path

Call Hamiltonian(k+1)

Remove it (Backtrack)

Validity Check (Easy)

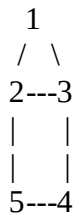
IsValid(vertex)



1. Check whether it is connected to the previous vertex.
2. Check whether it is already used.
3. If it is the last vertex,
check whether it connects to the first vertex.

Step-by-Step Example

Graph:



Step 1

Start from vertex **1**

Path = [1]

Step 2

Choose **2**

Path = [1,2]

Step 3

Choose **5**

Path = [1,2,5]

Step 4

Choose **4**

Path = [1,2,5,4]

Step 5

Choose **3**

Path = [1,2,5,4,3]

Step 6

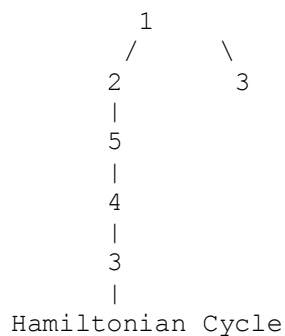
Check whether **3** is connected to **1**.

Yes ✓

So the Hamiltonian Cycle is:

$1 \rightarrow 2 \rightarrow 5 \rightarrow 4 \rightarrow 3 \rightarrow 1$

State Space Tree



Time Complexity

- Worst Case: $O(n!)$
- Space Complexity: $O(n)$



7. Explain the Heap data structure. Perform Heap Sort on the following elements and obtain the sorted order. Show all intermediate steps.

2, 9, 7, 6, 5, 8

Heap sort is one of the sorting algorithms used to arrange a list of elements in order. Heapsort algorithm uses one of the tree concepts called Heap Tree. In this sorting algorithm, we use Max Heap to arrange list of elements in Descending order and Min Heap to arrange list elements in Ascending order.

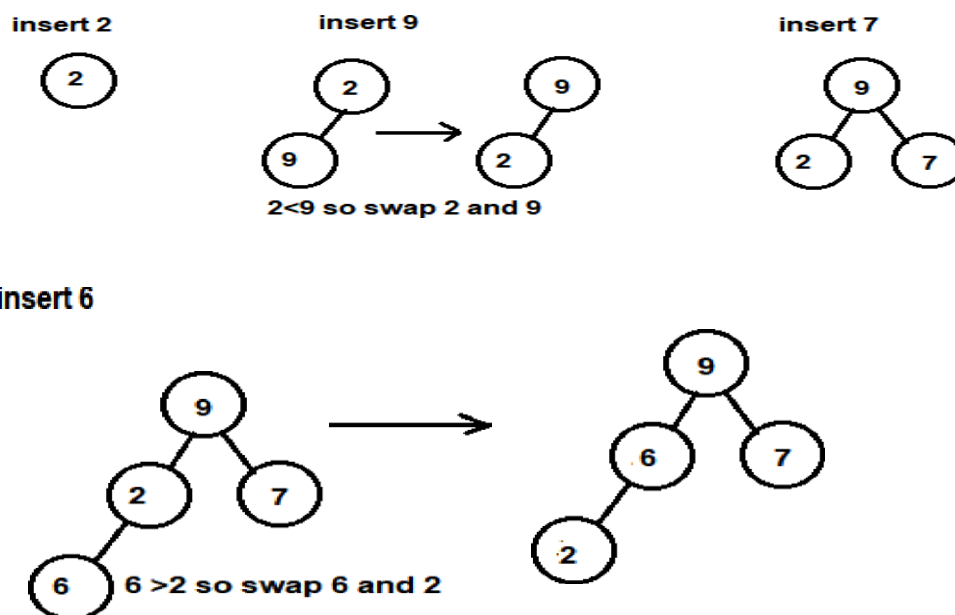
Heap sort algorithm is divided into two basic parts:

1. Creating a Heap of the unsorted list/array.
2. Then a sorted array is created by repeatedly removing the largest/smallest element from the heap, and inserting it into the array. The heap is reconstructed after each removal.

Max Heap

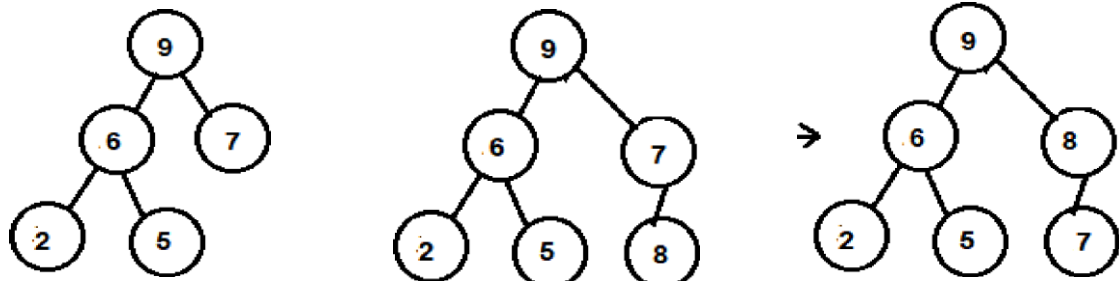
In max heap the parent node contains greater value than its children.

Construct a Max heap for the following list of elements: 2, 9, 7, 6, 5, 8

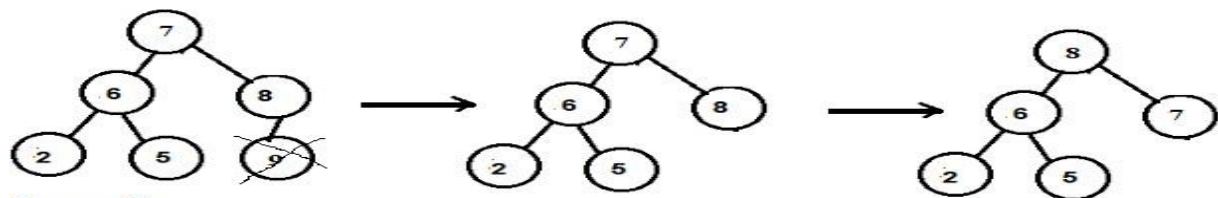




insert 5

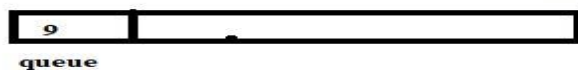


$8 > 7$ so swap 8 and 7



Remove 9

7 is Compared with 6 & 8.
 $8 > 7$. swap 7 and 8.

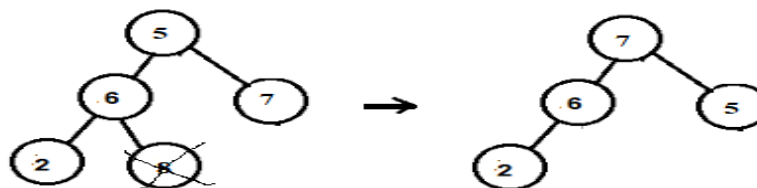


queue

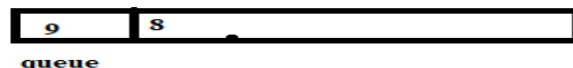
Deletion of Max Heap

Now Remove the largest element from binary heap by swapping it with the last leaf node. Here 9 is swapped with 7. after swapping remove 9. Now 7 is in root position compare 7 with its children(6 & 8). If parent is greater than children than nothing to do otherwise, select the largest value from children and swap it with the parent. Here 8 is greater value so swap 7 with 8.

Delete 8



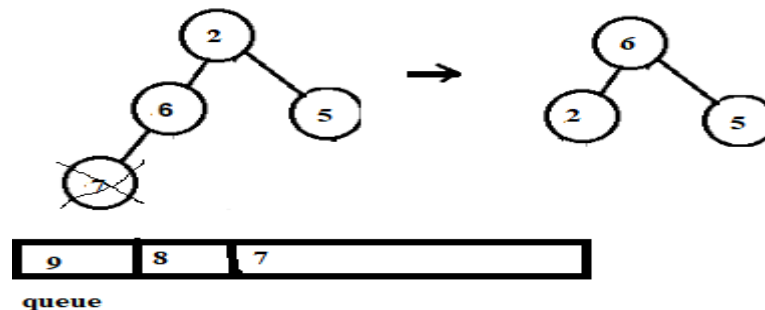
Delete 8 and Compare 5 with 6 & 7.
 $7 > 5$ swap 7 and 5



queue

Delete 7

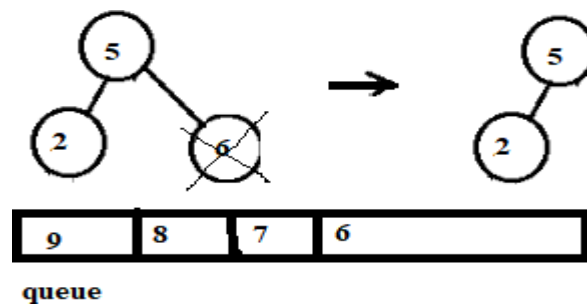
swap 7 with last leaf node 2. Delete 7 and add it to queue. Then compare 2 with 6 & 5. $6 > 2$ swap 6 and 2.



Delete 6

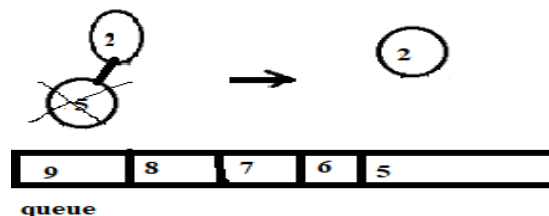
To delete 6 swap with last leaf node 5. Delete 6 and add it to queue. Then compare 5 and 2.

$5 > 2$ so no change in the tree.

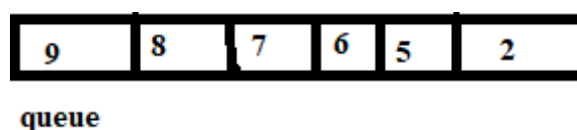


Delete 5

To delete 5 swap 5 with last leaf node 2, then delete 5 and add it in the queue. Now the tree has only one node, i.e. 2.



Delete 2—There is only one node (i.e. 2) is there delete that node and add it to queue.



In descending order the elements are sorted by using max heap sort.



Min Heap

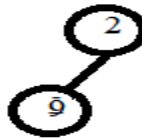
In min heap the parent node contains lesser value than its children nodes.

Construct a Min heap for the following list of elements: 2,9,7,6,5,8

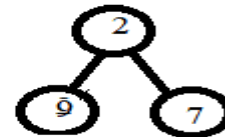
1.insert 2



2. Insert 9

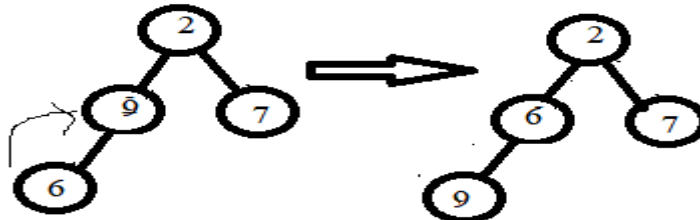


3. insert 7



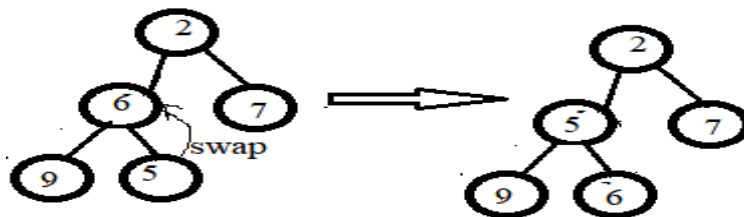
Insert 6. 6 is inserted as left child of 9. Now compare 6 with its parent 9. 6 is less than 9, so swap 6 and 9. After swapping compare 6 with its parent that is 2. 2 is less than 6 so no need to swap.

insert 6



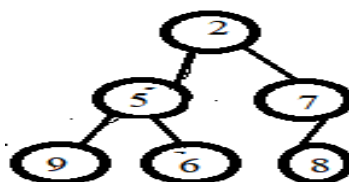
Insert 5. 5 is inserted as right child of 6. Now compare 5 with its parent node 6. 5 is less than 6 so swap 5 and 6. After Swapping compare 5 with its parent node 2. 2 is less than 5 no need to swap.

insert 5



Insert 8. 8 is inserted as left child of 7. Compare 8 with its parent node 7. 7 is less than 8.

insert 8



Now the binary heap is constructed.

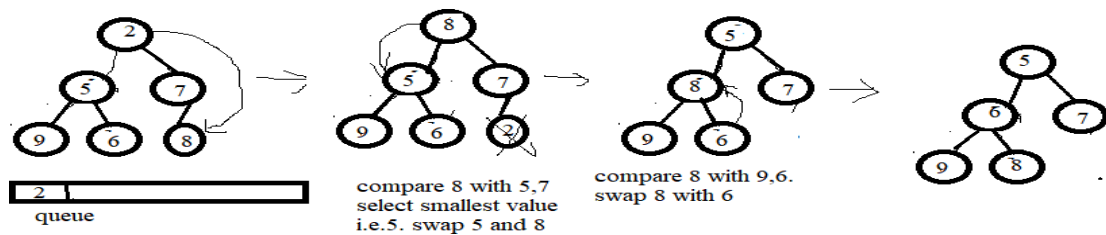
Deletion of Min Heap

Delete root node 2.

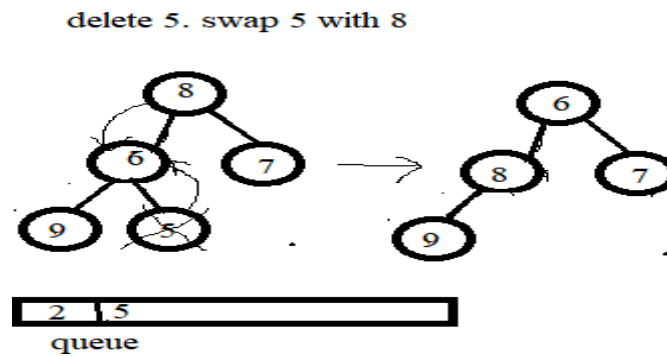


2 is swapped with last leaf node in the tree. Here the last leaf node is 8. After swapping delete node 2 and add it into the queue. Now compare new root node

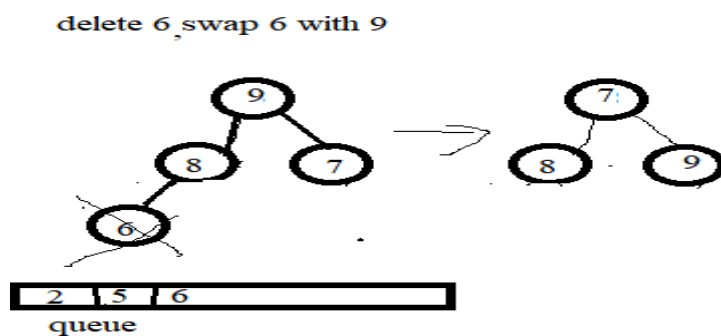
8 with its children 5 & 7. Select the smallest value(i.e 5). Now 5 becomes new root node.



Delete 5



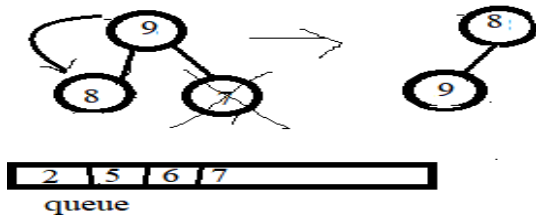
Delete 6





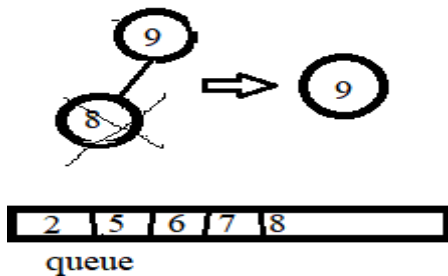
Delete 7

delete 7 swap it with 9



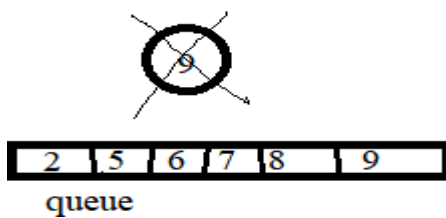
Delete 8

delete 8



Delete 9

delete 9



Finally the queue contains the elements in the ascending order.



8. Explain the Disjoint Set data structure in detail. Describe its operations with suitable examples.

Disjoint Sets:

The disjoint sets are those that do not have any common element.

For example $S1 = \{1,7,8,9\}$ and $S2 = \{2,5,10\}$, then we can say that $S1$ and $S2$ are two disjoint sets.

Disjoint Set Operations:

The disjoint set operations are

1. Union
2. Find

Disjoint set Union:

If S_i and S_j are two disjoint sets, then their union $S_i \cup S_j$ consists of all the elements x such that x is in S_i or S_j .

Example:

$S1 = \{1,7,8,9\}$ $S2 = \{2,5,10\}$

$S1 \cup S2 = \{1,2,5,7,8,9,10\}$

FIND: The **Simple Find** operation finds the root node (representative element) of the set containing a given element.

Given the element i , find the set containing i .

Example:

$S1 = \{1,7,8,9\}$ $S2 = \{2,5,10\}$ $S3 = \{3,4,6\}$

Then,

$\text{Find}(4) = S3$ $\text{Find}(5) = S2$ $\text{Find}(7) = S1$

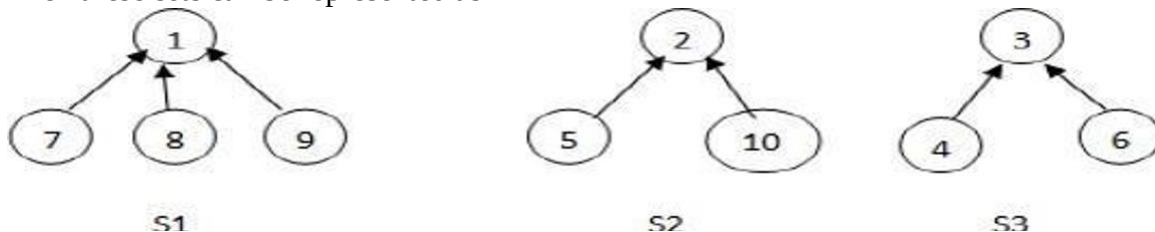
Set Representation:

The set will be represented as the tree structure where all children will store the address of parent / root node. The root node will store null at the place of parent address. In the given set of elements any element can be selected as the root node, generally we select the first node as the root node.

Example:

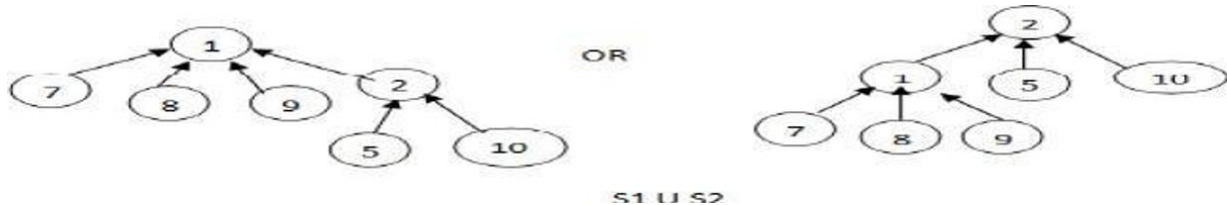
$S1 = \{1,7,8,9\}$ $S2 = \{2,5,10\}$ $S3 = \{3,4,6\}$

Then these sets can be represented as



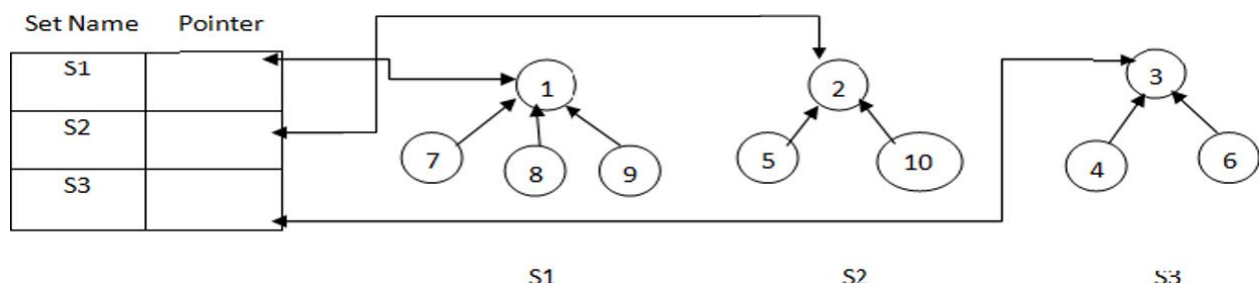
Disjoint Union:

To perform disjoint set union between two sets S_i and S_j can take any one root and make it sub-tree of the other. Consider the above example sets $S1$ and $S2$ then the union of $S1$ and $S2$ can be represented as any one of the following.



Find:

To perform find operation, along with the tree structure we need to maintain the name of each set. So, we require one more data structure to store the set names. The data structure contains two fields. One is the set name and the other one is the pointer to root.



UNION AND FIND ALGORITHMS:

Simple Union

Definition

The Simple Union operation combines two disjoint sets into a single set by making one root the parent of another root.

Rules for Simple Union

6. Consider all elements as individual sets.
7. Initialize the parent array with -1.
8. Select two root nodes.
9. Make one root the parent of the other.
10. Update the parent array after every union.

Algorithm

SimpleUnion(i, j)

```
{
  P[j] = i;
}
```

where

- i = root of first set
- j = root of second set

After union, i becomes the parent of j.



Example

Consider the elements

1 2 3 4 5 6

Step 1: Initial State

$P[1] = -1$

$P[2] = -1$

$P[3] = -1$

$P[4] = -1$

$P[5] = -1$

$P[6] = -1$

Tree Representation

1 2 3 4 5 6

Each element is a separate set.

Step 2: Union(1,2)

$P[2] = 1$

Tree

1

|

2

Parent Array

i	1	2	3	4	5	6
---	---	---	---	---	---	---

P[i]	-1	1	-1	-1	-1	-1
------	----	---	----	----	----	----

Step 3: Union(3,4)

$P[4] = 3$

Tree

3

|

4

Parent Array

i	1	2	3	4	5	6
---	---	---	---	---	---	---

P[i]	-1	1	-1	3	-1	-1
------	----	---	----	---	----	----

Step 4: Union(5,6)

$P[6] = 5$

Tree

5

|

6

Parent Array

i	1	2	3	4	5	6
---	---	---	---	---	---	---

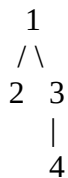
P[i]	-1	1	-1	3	-1	5
------	----	---	----	---	----	---



Step 5: Union(1,3)

$P[3] = 1$

Tree



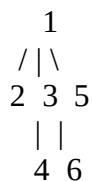
Parent Array

i	1	2	3	4	5	6
P[i]	-1	1	1	3	-1	5

Step 6: Union(1,5)

$P[5] = 1$

Final Tree



Final Parent Array

i	1	2	3	4	5	6
P[i]	-1	1	1	3	1	5

Simple Find

Definition

The **Simple Find** operation finds the root node (representative element) of the set containing a given element.

Algorithm

Find(i)

```
{
    while(P[i] > 0)
    {
        i = P[i];
    }

    return i;
}
```

Rules for Simple Find

6. Start from the given node.
7. Follow parent links.



8. Continue until a node with parent -1 is reached.

9. That node is the root.

10. Return the root.

Example

Find(4)

Step 1

$i = 4$

$P[4] = 3$

Move to parent.

$i = 3$

Step 2

$P[3] = 1$

Move to parent.

$i = 1$

Step 3

$P[1] = -1$

Root found.

Return 1

Tracing Table for Find(4)

Step	i	P[i]
1	4	3
2	3	1
3	1	-1

Therefore,

Find(4) = 1

Time Complexity

Operation Complexity

Simple Union $O(1)$

Simple Find $O(n)$ (Worst Case)



9. Explain the general method of Backtracking. Discuss its state-space tree in detail.

Definition: The backtracking algorithm is a problem-solving approach that tries out all the possible solutions and chooses the best or desired ones. Generally, it is used to solve problems that have multiple solutions.

Terminology Used in Backtracking

Root Node

Represents the initial state of the problem.

Live Node

A node that has been generated but not yet explored.

E-node (Expansion Node)

The live node currently being expanded.

Dead Node

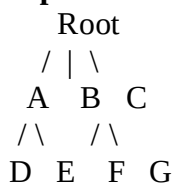
A node that cannot lead to a solution and is discarded.

Answer Node

A node representing a complete valid solution.

State Space Tree

A **State Space Tree** is a graphical representation of all possible states generated during the search.



- Root → Initial state
- Internal nodes → Partial solutions
- Leaf nodes → Complete solutions
- Invalid branches are pruned

Backtracking Algorithm

Backtrack(k)

```
{
  if solution found
    print solution
  else
    {
      for each candidate x
      {
        if x is promising
        {
          include x
          Backtrack(k+1)
          remove x
        }
      }
    }
}
```



```
}  
}
```

Steps in Backtracking

- 1 Start from the root node.
- 2 Generate a candidate solution.
- 3 Check whether it is promising.
- 4 If promising, continue searching.
- 5 If not promising, backtrack.
- 6 Repeat until all solutions are explored.

Applications of Backtracking

- 1 N-Queens Problem
- 2 Graph Coloring
- 3 Hamiltonian Cycle
- 4 Sum of Subsets

10. Compare the applications of Backtracking for Sum of Subsets and Graph Coloring problems.

Backtracking is a general algorithmic technique used to solve problems by constructing a solution step-by-step. At each step, the algorithm makes a possible choice and checks whether the choice satisfies the given constraints. If the choice cannot lead to a valid solution, the algorithm backtracks to the previous step and tries another choice.

Two important applications of backtracking are:

1. Sum of Subsets Problem
2. Graph Coloring Problem

Although both use the same backtracking principle, their choices, constraints, state-space trees, and applications are different.

1. Sum of Subsets Problem

Given a set of positive integers:

$$W = \{w_1, w_2, \dots, w_n\}$$

and a target value M , the Sum of Subsets problem finds all subsets whose sum is exactly equal to M .

2. Graph Coloring Problem

In the Graph Coloring problem, colors are assigned to the vertices of a graph such that **no two adjacent vertices have the same color**.

If at most m colors are available, the problem is called the **m-Coloring problem**.



Feature	Sum of Subsets	Graph Coloring
Problem objective	Find subsets whose sum equals a given target value	Assign colors to vertices so that adjacent vertices have different colors
Input	Set of positive numbers and target sum	Graph and number of available colors
Solution representation	Include/exclude each element	Assign a color to each vertex
Decision at each step	Include an element or exclude it	Select one of the available colors
Constraint checked	Current sum should not exceed target; remaining elements should allow target to be reached	A vertex must not have the same color as an already-colored adjacent vertex
Backtracking occurs when	Current subset cannot produce the target sum	No valid color can be assigned to the current vertex
Typical state-space tree	Binary tree: Include / Exclude	Multi-way tree: Color 1 / Color 2 / ... / Color m
Example application	Selecting items whose weights/costs add to a required value	Assigning exam slots so that conflicting subjects do not get the same slot
Type of constraint	Numerical constraint	Graph/adjacency constraint



MODULE – III

Short Answer Questions

1. What is Dynamic Programming?

Dynamic Programming (DP) is an algorithm design technique used to solve problems by dividing them into overlapping subproblems, solving each subproblem once, and storing its result for future use.

2. What are the two important properties of Dynamic Programming?

1. Optimal substructure – an optimal solution contains optimal solutions to its subproblems.
2. Overlapping subproblems – the same subproblems are solved repeatedly.

3. What are the two approaches used in Dynamic Programming?

- Top-down approach: Uses recursion with memoization.
- Bottom-up approach: Solves smaller subproblems first and builds the final solution iteratively.

4. What is memoization?

Memoization is a top-down DP technique in which the results of already solved subproblems are stored so they do not have to be computed again.

5. What is tabulation?

Tabulation is a bottom-up DP technique that stores solutions of smaller subproblems in a table and uses them to obtain the final solution.

6. What is an Optimal Binary Search Tree?

An Optimal Binary Search Tree (OBST) is a binary search tree constructed so that the expected search cost is minimum, considering the search probabilities of the keys.

7. What is the objective of the 0/1 Knapsack problem?

The objective is to select a subset of items that maximizes the total profit without exceeding the given knapsack capacity. Each item can either be selected once or not selected.

8. What is the All Pairs Shortest Path problem?

The All Pairs Shortest Path problem finds the shortest path between every pair of vertices in a weighted graph.

- It calculates the minimum distance from each vertex to all other vertices.
- Floyd–Warshall algorithm is commonly used to solve All pairs shortest path problem.



- It can handle directed and weighted graphs

9. What is the Traveling Salesperson Problem (TSP)?

The Traveling Salesperson Problem is the problem of finding the minimum-cost tour in which a salesperson:

- Starts from a city,
- Visits every other city exactly once, and
- Returns to the starting city.

10. What is Reliability Design?

Reliability Design is the process of designing a system to achieve maximum reliability while satisfying a given cost or resource constraint.

- Multiple components may be used to improve system reliability.
- Dynamic Programming can be used to find the optimal number of components.
- The objective is to obtain high reliability at minimum cost.



Long Answer Questions

1. Explain the Dynamic Programming technique in detail. Discuss its general method and the principle of optimality.

Dynamic Programming is an algorithmic technique used to solve optimization and decision problems by breaking them into smaller overlapping subproblems, solving each subproblem only once, storing the results, and reusing them to obtain the optimal solution efficiently.

The two key properties required to apply **Dynamic Programming** are:

1. **Optimal Substructure:** The optimal solution to a problem can be obtained by combining the optimal solutions of its subproblems.
2. **Overlapping Subproblems:** The same subproblems are solved repeatedly, so their solutions are stored and reused to avoid recomputation.

Steps of Dynamic Programming Approach Dynamic Programming algorithm is designed using the following four steps –

1. Characterize the structure of an optimal solution.
2. Recursively define the value of an optimal solution
3. Compute the value of an optimal solution, typically in a bottom-up fashion.
4. Construct an optimal solution from the computed information.

Control Abstraction for Dynamic Programming

1. Principle of Optimality :-An optimal sequence of decisions has the property that whatever the initial state and decisions are, the remaining decisions must constitute an optimal decision sequence with regard to the state resulting from the first decision,
2. Optimal substructure: For the optimal solution of the problem, a solution of sub problem also must be optimal. Dynamic programming builds the optimal solution of the bigger problem using the solution of smaller sub problems. Hence we should consider only those sub problems which have an optimal solution.
3. Overlapping subproblems: When the big problem is divided into small problems, it may create exponential subproblems.

Dynamic Programming algorithm is designed using the following four steps:-

- 1.Characterize the structure of an optimal solution.
2. Recursively define the value of an optimal solution.
- 3.Compute the value of an optimal solution, typically in a bottom-up fashion.
4. Construct an optimal solution from the computed information.
 - Dynamic programming (DP) splits the large problem at every possible point. When the problem becomes sufficiently small, DP solves it.
 - Dynamic programming is bottom up approach, it finds the solution of the smallest problem and constructs the solution of the larger problem from already solved smaller problems.
 - To avoid recomputation of the same problem, DP saves the result of sub problems into the table. When next time same problem encounters, the answer is retrieved from the table by lookup procedure.



2. Differentiate between Dynamic Programming and Divide and Conquer.

Divide & Conquer (D&C)	Dynamic Programming (DP)
Divides a problem into smaller independent subproblems.	Divides a problem into overlapping (interdependent) subproblems.
Follows three steps : 1. Divide 2. Conquer 3. Combine.	Follows four steps : 1. Characterize optimal solution 2. Define recurrence relation . Compute solutions (bottom-up or memoization) 4. Construct optimal solution.
Uses recursion .	Usually uses iteration (bottom-up) , though it can also use recursion with memoization.
Solves the same subproblem repeatedly if it appears multiple times.	Solves each subproblem only once and stores the result in a table (memoization/tabulation).
Uses a top-down approach.	Mainly uses a bottom-up approach (or top-down with memoization).
Subproblems are independent of each other.	Subproblems are overlapping/interdependent .
Does not store intermediate results.	Stores intermediate results to avoid repeated computation.
May perform redundant computations.	Eliminates redundant computations, making it more efficient for overlapping subproblems.
Generally simpler to design.	Requires identifying optimal substructure and overlapping subproblems.
Time complexity may be higher when subproblems overlap.	Usually reduces time complexity by reusing previously computed results.
Suitable for problems that can be split into independent parts.	Suitable for optimization problems with overlapping subproblems.
Examples: Merge Sort, Quick Sort, Binary Search, Strassen's Matrix Multiplication.	Examples: Optimal binary search tree, 0/1 Knapsack, All-Pairs Shortest Path (Floyd–Warshall). Traveling salesperson problem



3.Explain the control abstraction of Dynamic Programming. Write the general control abstraction Algorithm.

Control Abstraction for Dynamic Programing

1. Principle of Optimality :-An optimal sequence of decisions has the property that whatever the initial state and decisions are, the remaining decisions must constitute an optimal decision sequence with regard to the state resulting from the first decision,
2. Optimal substructure: For the optimal solution of the problem, a solution of sub problem also must be optimal. Dynamic programming builds the optimal solution of the bigger problem using the solution of smaller sub problems. Hence we should consider only those sub problems which have an optimal solution.
3. Overlapping subproblems: When the big problem is divided into small problems, it may create exponential subproblems.

Dynamic Programming algorithm is designed using the following four steps:-

- 1.Characterize the structure of an optimal solution.
 2. Recursively define the value of an optimal solution.
 - 3.Compute the value of an optimal solution, typically in a bottom-up fashion.
 4. Construct an optimal solution from the computed information.
- Dynamic programming (DP) splits the large problem at every possible point. When the problem becomes sufficiently small, DP solves it.
 - Dynamic programming is bottom up approach, it finds the solution of the smallest problem and constructs the solution of the larger problem from already solved smaller problems.
 - To avoid recomputation of the same problem, DP saves the result of sub problems into the table. When next time same problem encounters, the answer is retrieved from the table by lookup procedure.



```
Algorithm DYNAMIC_PROGRAMMING (P)

if solved(P) then
    return lookup(P)
else
    Ans ← SOLVE(P)
    store (P, Ans)
end

FUNCTION SOLVE(P)
    if sufficiently small(P) then
        solution(P)      // Find solution for sufficiently small problem
    else
        Divide P into smaller sub problems P1, P2, ..., Pn
        Ans1 ← DYNAMIC_PROGRAMMIN(P1)
        Ans2 ← DYNAMIC_PROGRAMMIN(P2)
        .
        .
        Ansn ← DYNAMIC_PROGRAMMIN(Pn)
        return (combine(Ans1, Ans2 ..., Ansn))
    end
```



4. Consider a 0/1 Knapsack Problem with $n = 4$ items having weights $w_1=2$, $w_2=3$, $w_3=4$, and $w_4=5$ and corresponding profits $p_1=3$, $p_2=4$, $p_3=5$, and $p_4=6$. If the knapsack capacity is $W = 5$, determine the maximum profit and the selected items.

Given:

Item (i)	Profit (p_i)	Weight (w_i)
1	3	2
2	4	3
3	5	4
4	6	5

Given

- Number of items, $n = 4$
- Capacity, $M = W = 5$

Step 1: Apply the Formula and Build the Sequence of Decisions

Initially,

$$S^0 = \{(0,0)\}$$

where each pair represents **(Profit, Weight)**.

For $i = 0$ (Item 1)

$$S_1^0 = \{(3,2)\}$$

Merge

$$S^1 = S^0 \cup S_1^0$$

$$S^1 = \{(0,0), (3,2)\}$$

For $i = 1$ (Item 2)

Current

$$S^1 = \{(0,0), (3,2)\}$$

Generate

$$S_1^1 = \{(4,3), (7,5)\}$$

Merge

$$S^2 = S^1 \cup S_1^1$$

$$S^2 = \{(0,0), (3,2), (4,3), (7,5)\}$$



(No redundant pairs.)

For i = 2 (Item 3)

Current

$$S^2 = \{(0,0), (3,2), (4,3), (7,5)\}$$

Generate

$$S_1^2 = \{(5,4), (8,6), (9,7), (12,9)\}$$

Since capacity $M = 5$,

Remove

$$(8,6), (9,7), (12,9)$$

Remaining

$$S_1^2 = \{(5,4)\}$$

Merge

$$S^3 = S^2 \cup S_1^2$$

$$S^3 = \{(0,0), (3,2), (4,3), (5,4), (7,5)\}$$

For i = 3 (Item 4)

Current

$$S^3 = \{(0,0), (3,2), (4,3), (5,4), (7,5)\}$$

Generate

$$S_1^3 = \{(6,5), (9,7), (10,8), (11,9), (13,10)\}$$

Remove pairs whose weight exceeds 5.

Remaining

$$S_1^3 = \{(6,5)\}$$

Merge

$$S^4 = S^3 \cup S_1^3$$

Since both **(6,5)** and **(7,5)** have the same weight, keep the one with **maximum profit**.
Therefore,

$$S^4 = \{(0,0), (3,2), (4,3), (5,4), (7,5)\}$$

Step 2: Find the Optimal Solution

The maximum feasible pair is

$$(7,5)$$

Hence,

- Maximum Profit = 7
- Total Weight = 5

Step 3: Trace Back the Solution

The optimal pair

$$(7,5) \in S^4$$

Since



$$(7,5) \in S^3$$

Item 4 is **not selected**.

$$x_4 = 0$$

Now,

$$(7,5) \in S^3$$

Subtract Item 3:

$$(7 - 5, 5 - 4) = (2, 1)$$

This pair is **not present** in S^2 .

Hence,

$$x_3 = 0$$

Now,

$$(7,5) \in S^2$$

Subtract Item 2:

$$(7 - 4, 5 - 3) = (3, 2)$$

Since

$$(3, 2) \in S^1, x_2 = 1$$

Now,

$$(3, 2) \in S^1$$

Subtract Item 1:

$$(3 - 3, 2 - 2) = (0, 0)$$

Since

$$(0, 0) \in S^0, x_1 = 1$$

Final Solution

$$(x_1, x_2, x_3, x_4) = (1, 1, 0, 0)$$

Selected Items:

Item Profit Weight

1	3	2
2	4	3

Total Weight

$$2 + 3 = 5$$

Total Profit

$$3 + 4 = 7$$

Answer

- **Optimal State:** $(7,5)$
- **Maximum Profit:** 7
- **Selected Items:** Item 1 and Item 2



- **Decision Vector:** $(1,1,0,0)$

5. Solve a Reliability Design problem using Dynamic Programming for a system with multiple stages and alternative component choices and determine the design with maximum reliability.

Reliability design is the problem of selecting the best components and adding redundancy to maximize the reliability of a system within a given cost or resource limit.

Reliability design means improving the reliability of a system by adding redundant components, while staying within a given budget.

Example

A system has two subsystems connected in series with component reliabilities 0.8 and 0.7, costing ₹10 and ₹15 per component, respectively. If the total budget is ₹40, determine the optimal reliability design using Dynamic Programming by adding parallel redundant components.

A system has 2 subsystems connected in series.

Subsystem	One component reliability	Cost of one component
1	0.8	₹10
2	0.7	₹15

Assume we can add parallel redundant components. The total budget is ₹40.

Step 1: Reliability of parallel components

If a component has reliability r , and m identical components are connected in parallel:

$$R(m) = 1 - (1 - r)^m$$

Step 2: Possible choices

Subsystem 1

- 1 component:

$$R_1 = 0.8, \quad \text{Cost} = 10$$

- 2 parallel components:

$$R_1 = 1 - (1 - 0.8)^2 = 1 - (0.2)^2 = 0.96$$

$$\text{Cost} = ₹20.$$

Subsystem 2

- 1 component:

$$R_2 = 0.7, \quad \text{Cost} = 15$$

- 2 parallel components:

$$R_2 = 1 - (1 - 0.7)^2 = 1 - (0.3)^2 = 0.91$$

$$\text{Cost} = ₹30.$$



Step 3: Check all combinations

Subsystem 1	Subsystem 2	Total Cost	System Reliability
1 component	1 component	$10 + 15 = 25$	$0.8 \times 0.7 = 0.56$
2 components	1 component	$20 + 15 = 35$	$0.96 \times 0.7 = 0.672$
1 component	2 components	$10 + 30 = 40$	$0.8 \times 0.91 = 0.728$
2 components	2 components	$20 + 30 = 50$	Not possible

Optimal solution

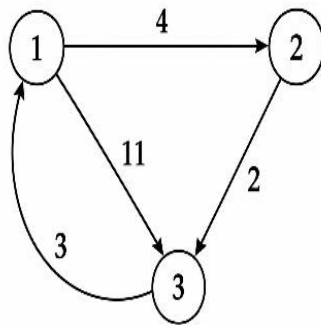
For a budget of ₹40, the best choice is:

- Subsystem 1 → **1 component**
- Subsystem 2 → **2 parallel components**

$Maximum Reliability = 0.728$



6. Compute All pairs shortest path problem for following graph



Step 1: Construct A^0 Matrix

First, we construct the initial matrix A^0 from the given graph.

Statement:

Initially, no intermediate vertex is considered. Therefore, A^0 contains the direct edge weights between the vertices. The diagonal elements are 0 because the distance from a vertex to itself is 0. If there is no direct edge between two vertices, its value is taken as ∞ .

Therefore,

$$A^0 = \begin{array}{c|ccc} & 1 & 2 & 3 \\ \hline 1 & 0 & 4 & 11 \\ 2 & 6 & 0 & 2 \\ 3 & 3 & \infty & 0 \end{array}$$

Here:

- $A^0[1,2] = 4$
- $A^0[1,3] = 11$
- $A^0[2,1] = 6$
- $A^0[2,3] = 2$
- $A^0[3,1] = 3$
- $A^0[3,2] = \infty$

Step 2: Construct A^1 Matrix

Now, **vertex 1** is considered as the intermediate vertex.

Statement:



From A^0 , we construct A^1 by considering vertex 1 as the intermediate vertex. For every pair of vertices i and j , we check whether the path through vertex 1 is shorter than the existing path.

The formula is:

$$A^{(k)}[i][j] = \min(A^{(k-1)}[i][j], A^{(k-1)}[i][k] + A^{(k-1)}[k][j])$$

Structural Breakdown

$$A^1[i,j] = \min (A^0[i,j], A^0[i,1] + A^0[1,j])$$

Find $A^1[2,3]$

We check whether going from **2 to 3 through vertex 1** gives a shorter path.

$$\begin{aligned} A^1[2,3] &= \min (A^0[2,3], A^0[2,1] + A^0[1,3]) \\ &= \min (2, 6 + 11) \\ &= \min (2, 17) \\ &= 2 \end{aligned}$$

Therefore,

$$A^1[2,3] = 2$$

Find $A^1[3,2]$

There is no direct path from 3 to 2.

Therefore,

$$\begin{aligned} A^1[3,2] &= \min (A^0[3,2], A^0[3,1] + A^0[1,2]) \\ &= \min (\infty, 3 + 4) \\ &= \min (\infty, 7) \\ &= 7 \end{aligned}$$

Therefore,

$$A^1[3,2] = 7$$

Hence,

	1	2	3
1	0	4	11
2	6	0	2
3	3	7	0



When vertex 1 is considered as the intermediate vertex, the first row and first column remain unchanged.

Step 3: Construct A^2 Matrix

Now, **vertex 2 is considered as the intermediate vertex.**

Statement:

From A^1 , we construct A^2 by considering vertex 2 as the intermediate vertex. We check whether passing through vertex 2 gives a shorter path between every pair of vertices.

The formula is:

$$A^2[i,j] = \min (A^1[i,j], A^1[i,2] + A^1[2,j])$$

Find $A^2[1,3]$

Therefore,

$$\begin{aligned} A^2[1,3] &= \min (A^1[1,3], A^1[1,2] + A^1[2,3]) \\ &= \min (11, 4 + 2) \\ &= \min (11, 6) \\ &= 6 \end{aligned}$$

Therefore,

$$A^2[1,3] = 6$$

Find $A^2[3,1]$

We check whether going from 3 to 1 through vertex 2 gives a shorter path.

$$\begin{aligned} A^2[3,1] &= \min (A^1[3,1], A^1[3,2] + A^1[2,1]) \\ &= \min (3, 7 + 6) \\ &= \min (3, 13) \\ &= 3 \end{aligned}$$

Therefore,

$$A^2[3,1] = 3$$



Hence,

	1	2	3
1	0	4	6
2	6	0	2
3	3	7	0

Step 4: Construct A^3 Matrix

Now, **vertex 3 is considered as the intermediate vertex.**

Statement:

From A^2 , we construct A^3 by considering vertex 3 as the intermediate vertex. For every pair of vertices, we check whether passing through vertex 3 produces a shorter path.

The formula is:

$$A^3[i,j] = \min (A^2[i,j], A^2[i,3] + A^2[3,j])$$

Find $A^3[1,2]$

Therefore,

$$\begin{aligned} A^3[1,2] &= \min (A^2[1,2], A^2[1,3] + A^2[3,2]) \\ &= \min (4, 6 + 7) \\ &= \min (4, 13) \\ &= 4 \end{aligned}$$

Therefore,

$$A^3[1,2] = 4$$

Find $A^3[2,1]$

Therefore,

$$\begin{aligned} A^3[2,1] &= \min (A^2[2,1], A^2[2,3] + A^2[3,1]) \\ &= \min (6, 2 + 3) \\ &= \min (6, 5) \\ &= 5 \end{aligned}$$



Therefore,

$$A^3[2,1] = 5$$

Final A^3 Matrix

$$A^3 = \begin{bmatrix} 0 & 4 & 6 \\ 5 & 0 & 2 \\ 3 & 7 & 0 \end{bmatrix}$$

This is the **final All Pairs Shortest Path matrix**.

Time Complexity

There are three nested loops:

$k=1$ to n

$i=1$ to n

$j=1$ to n

Therefore, the number of operations is approximately:

$$n \times n \times n = n^3$$

Hence, Time Complexity: $O(n^3)$

Space Complexity

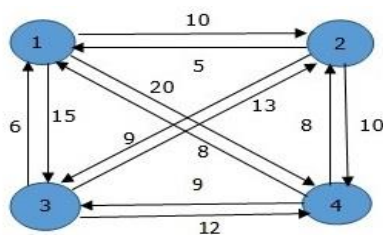
The algorithm uses an $n \times n$ distance matrix.

Therefore,

Space Complexity: $O(n^2)$



7. Solve the Travelling sales Persons problem using Dynamic Programming. and determine the minimum-cost tour for below given graph.



The distance matrix is:

From/To	1	2	3	4
1	0	10	15	20
2	5	0	9	10
3	6	13	0	12
4	8	8	9	0

The solution is calculated in stages according to the number of remaining cities.

Step 1: $S = \phi$

No cities are left, so return to city 1.

$$Cost(2, \phi, 1) = d(2,1) = 5$$

$$Cost(3, \phi, 1) = d(3,1) = 6$$

$$Cost(4, \phi, 1) = d(4,1) = 8$$

Step 2: One City Remaining

From City 2

$$Cost(2, \{3\}, 1) = d(2,3) + Cost(3, \phi, 1) = 9 + 6 = 15$$

$$Cost(2, \{4\}, 1) = d(2,4) + Cost(4, \phi, 1) = 10 + 8 = 18$$

From City 3

$$Cost(3, \{2\}, 1) = d(3,2) + Cost(2, \phi, 1) = 13 + 5 = 18$$

$$Cost(3, \{4\}, 1) = d(3,4) + Cost(4, \phi, 1) = 12 + 8 = 20$$



From City 4

$$Cost(4, \{3\}, 1) = d(4, 3) + Cost(3, \phi, 1) = 9 + 6 = 15$$

$$Cost(4, \{2\}, 1) = d(4, 2) + Cost(2, \phi, 1) = 8 + 5 = 13$$

These intermediate values match the dynamic programming calculations in the source.

Step 3: Two Cities Remaining

From City 2

$$Cost(2, \{3, 4\}, 1) = \min \begin{cases} d(2, 3) + Cost(3, \{4\}, 1) \\ d(2, 4) + Cost(4, \{3\}, 1) \end{cases} = \min\{10 + 20, 10 + 15\} = \min\{35, 25\} = 25$$

From City 3

$$Cost(3, \{2, 4\}, 1) = \min \begin{cases} d(3, 2) + Cost(2, \{4\}, 1) \\ d(3, 4) + Cost(4, \{2\}, 1) \end{cases} = \min\{13 + 18, 12 + 13\} = \min\{31, 25\} = 25$$

From City 4

$$Cost(4, \{2, 3\}, 1) = \min \begin{cases} d(4, 2) + Cost(2, \{3\}, 1) \\ d(4, 3) + Cost(3, \{2\}, 1) \end{cases} = \min\{8 + 15, 9 + 18\} = \min\{23, 27\} = 23$$

Step 4: Final Calculation

Now start from city 1:

$$Cost(1, \{2, 3, 4\}, 1) = \min \begin{cases} d(1, 2) + Cost(2, \{3, 4\}, 1) \\ d(1, 3) + Cost(3, \{2, 4\}, 1) \\ d(1, 4) + Cost(4, \{2, 3\}, 1) \end{cases} = \min\{10 + 25, 15 + 25, 20 + 23\} = \min\{35, 40, 43\}$$

Therefore, the **minimum cost is 35**.

Optimal Tour

$$\boxed{1 \rightarrow 2 \rightarrow 4 \rightarrow 3 \rightarrow 1}$$

Verification

$$1 \rightarrow 2 = 10 \quad 2 \rightarrow 4 = 10 \quad 4 \rightarrow 3 = 9 \quad 3 \rightarrow 1 = 6 \quad \text{Total Cost} = 10 + 10 + 9 + 6 = \boxed{35}$$



8. What are overlapping subproblems and optimal substructure? Explain these properties in detail and discuss their importance in Dynamic Programming.

Dynamic Programming is an algorithmic technique used to solve optimization and decision problems by breaking them into smaller overlapping subproblems, solving each subproblem only once, storing the results, and reusing them to obtain the optimal solution efficiently.

The two key properties required to apply **Dynamic Programming** are:

3. **Optimal Substructure:** The optimal solution to a problem can be obtained by combining the optimal solutions of its subproblems.
4. **Overlapping Subproblems:** The same subproblems are solved repeatedly, so their solutions are stored and reused to avoid recomputation.

Steps of Dynamic Programming Approach Dynamic Programming algorithm is designed using the following four steps –

1. Characterize the structure of an optimal solution.
2. Recursively define the value of an optimal solution
3. Compute the value of an optimal solution, typically in a bottom-up fashion.
4. Construct an optimal solution from the computed information.

Control Abstraction for Dynamic Programming

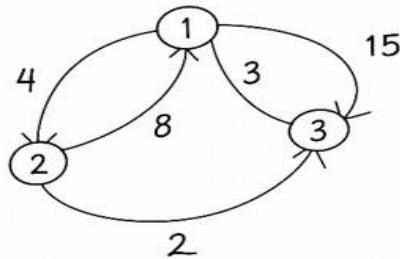
4. Principle of Optimality :-An optimal sequence of decisions has the property that whatever the initial state and decisions are, the remaining decisions must constitute an optimal decision sequence with regard to the state resulting from the first decision,
5. Optimal substructure: For the optimal solution of the problem, a solution of sub problem also must be optimal. Dynamic programming builds the optimal solution of the bigger problem using the solution of smaller sub problems. Hence we should consider only those sub problems which have an optimal solution.
6. Overlapping subproblems: When the big problem is divided into small problems, it may create exponential subproblems.

Dynamic Programming algorithm is designed using the following four steps:-

- 1.Characterize the structure of an optimal solution.
2. Recursively define the value of an optimal solution.
- 3.Compute the value of an optimal solution, typically in a bottom-up fashion.
4. Construct an optimal solution from the computed information.
 - Dynamic programming (DP) splits the large problem at every possible point. When the problem becomes sufficiently small, DP solves it.
 - Dynamic programming is bottom up approach, it finds the solution of the smallest problem and constructs the solution of the larger problem from already solved smaller problems.
 - To avoid recomputation of the same problem, DP saves the result of sub problems into the table. When next time same problem encounters, the answer is retrieved from the table by lookup procedure.



9. Compute All pairs shortest path problem for following graph



Step 1: Construct A^0 Matrix

First, we construct the initial matrix A^0 from the given graph.

Statement:

Initially, no intermediate vertex is considered. Therefore, A^0 contains the direct edge weights between the vertices.

- The diagonal elements are 0, because the distance from a vertex to itself is 0.
- If there is no direct edge between two vertices, its value is taken as ∞ .

Therefore,

$$A^0 = \begin{bmatrix} 0 & 4 & 11 \\ 6 & 0 & 2 \\ 3 & \infty & 0 \end{bmatrix}$$

Step 2: Construct A^1 Matrix

Now, vertex 1 is considered as the intermediate vertex.

Statement:

From A^0 , we construct A^1 by considering vertex 1 as the intermediate vertex.

For every pair of vertices i and j , we check whether the path through vertex 1 is shorter than the existing path.

The formula is: $A^{(k)}[i][j] = \min(A^{(k-1)}[i][j], A^{(k-1)}[i][k] + A^{(k-1)}[k][j])$

$$A^1[i][j] = \min(A^0[i][j], A^0[i][1] + A^0[1][j])$$

Find $A^1[2, 3]$

We check whether going from 2 to 3 through vertex 1 gives a shorter path.

$$\begin{aligned} A^1[2, 3] &= \min(A^0[2, 3], A^0[2, 1] + A^0[1, 3]) \\ &= \min(2, 6 + 11) \\ &= \min(2, 17) \\ &= \boxed{2} \end{aligned}$$

Therefore,



$$A^1[2, 3] = 2$$

Find $A^1[3, 2]$

$$\begin{aligned} A^1[3, 2] &= \min(A^0[3, 2]A^0[3, 1] + A^0[1, 2]) \\ &= \min(\infty, 3 + 4) \\ &= \min(\infty, 7) \\ &= \boxed{7} \end{aligned}$$

Therefore,

$$A^1[3, 2] = 7$$

$$A^1 = \begin{bmatrix} 0 & 4 & 11 \\ 6 & 0 & 2 \\ 3 & 7 & 0 \end{bmatrix}$$

Step 3: Construct A^2 Matrix

Now, vertex 2 is considered as the intermediate vertex.

Statement:

From A^1 , we construct A^2 by considering vertex 2 as the intermediate

The formula is:

$$A^2[i, j] = \min(A^1[i, 2]A^1[2, j] + A^1[i, j])$$

Find $A^2[1, 3]$

We check whether going from 1 to 3 through vertex 2 gives a shorter path.

$$\begin{aligned} A^2[1, 3] &= \min(A^1[1, 2]A^1[2, 3] + A^1[1, 3]) \\ &= \min(11, 4 + 2) \\ &= \min(11, 6) \\ &= \boxed{6} \end{aligned}$$

Therefore,

$$A^2[1, 3] = 6$$

with cost:

$$4 + 2 = 6$$

Find $A^2[3, 1]$

We check whether going from 3 to 1 through vertex 2 gives a shorter path.

$$\begin{aligned} A^2[3, 1] &= \min(A^1[3, 2]A^1[2, 1] + A^1[3, 1]) \\ &= \min(3, 7 + 6) \\ &= \min(3, 13) \\ &= \boxed{3} \end{aligned}$$



Therefore,

$$A^2[3, 1] = 3$$

Hence,

$$A^2 = \begin{bmatrix} 0 & 4 & 6 \\ 6 & 0 & 2 \\ 3 & 7 & 0 \end{bmatrix}$$

Step 4: Construct A^3 Matrix

Now, vertex 3 is considered as the intermediate vertex.

Statement:

From A^2 , we construct A^3 by considering vertex 3 as the intermediate vertex.

For every pair of vertices, we check whether passing through vertex 3 produces a shorter path.

The formula is:

$$A^3[i, j] = \min(A^2, [i, j]A^2[i, 3] + A^2[3, j])$$

Find $A^3[1, 2]$

We check whether going from 1 to 2 through vertex 3 gives a shorter path.

$$\begin{aligned} A^3[1, 2] &= \min(A^2, [1, 2]A^2[1, 3] + A^2[3, 2]) \\ &= \min(4, 6 + 7) \\ &= \min(4, 13) \\ &= \boxed{4} \end{aligned}$$

Therefore,

$$A^3[1, 2] = 4$$

Find $A^3[2, 1]$

We check whether going from 2 to 1 through vertex 3 gives a shorter path.

$$\begin{aligned} A^3[2, 1] &= \min(A^2, [2, 1]A^2[2, 3] + A^2[3, 1]) \\ &= \min(6, 2 + 3) \\ &= \min(6, 5) \\ &= \boxed{5} \end{aligned}$$

Therefore,

$$A^3[2, 1] = 5$$

with cost:

$$2 + 3 = 5$$

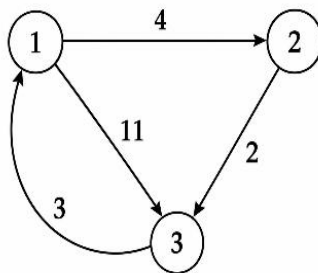


Final A^3 Matrix

Therefore, the final matrix is:

$$\begin{bmatrix} 0 & 4 & 6 \\ 5 & 0 & 2 \\ 3 & 7 & 0 \end{bmatrix}$$

10. Explain the working of the Floyd–Warshall algorithm with a suitable example.



Step 1: Construct A^0 Matrix

First, we construct the initial matrix A^0 from the given graph.

Statement:

Initially, no intermediate vertex is considered. Therefore, A^0 contains the direct edge weights between the vertices. The diagonal elements are 0 because the distance from a vertex to itself is 0. If there is no direct edge between two vertices, its value is taken as ∞ .

Therefore,

$$A^0 = \begin{array}{c|ccc} & 1 & 2 & 3 \\ \hline 1 & 0 & 4 & 11 \\ 2 & 6 & 0 & 2 \\ 3 & 3 & \infty & 0 \end{array}$$

Here:

- $A^0[1,2] = 4$
- $A^0[1,3] = 11$
- $A^0[2,1] = 6$



- $A^0[2,3] = 2$
- $A^0[3,1] = 3$
- $A^0[3,2] = \infty$

Step 2: Construct A^1 Matrix

Now, **vertex 1** is considered as the intermediate vertex.

Statement:

From A^0 , we construct A^1 by considering vertex 1 as the intermediate vertex. For every pair of vertices i and j , we check whether the path through vertex 1 is shorter than the existing path.

The formula is:

$$A^{(k)}[i][j] = \min(A^{(k-1)}[i][j], A^{(k-1)}[i][k] + A^{(k-1)}[k][j])$$

Structural Breakdown

$$A^1[i,j] = \min (A^0[i,j], A^0[i,1] + A^0[1,j])$$

Find $A^1[2,3]$

We check whether going from **2 to 3 through vertex 1** gives a shorter path.

$$\begin{aligned} A^1[2,3] &= \min (A^0[2,3], A^0[2,1] + A^0[1,3]) \\ &= \min (2, 6 + 11) \\ &= \min (2, 17) \\ &= 2 \end{aligned}$$

Therefore,

$$A^1[2,3] = 2$$

Find $A^1[3,2]$

There is no direct path from 3 to 2.

Therefore,

$$\begin{aligned} A^1[3,2] &= \min (A^0[3,2], A^0[3,1] + A^0[1,2]) \\ &= \min (\infty, 3 + 4) \\ &= \min (\infty, 7) \\ &= 7 \end{aligned}$$

Therefore,



$$A^1[3,2] = 7$$

Hence,

	1	2	3
1	0	4	11
2	6	0	2
3	3	7	0

When vertex 1 is considered as the intermediate vertex, the first row and first column remain unchanged.

Step 3: Construct A^2 Matrix

Now, vertex 2 is considered as the intermediate vertex.

Statement:

From A^1 , we construct A^2 by considering vertex 2 as the intermediate vertex. We check whether passing through vertex 2 gives a shorter path between every pair of vertices.

The formula is:

$$A^2[i,j] = \min (A^1[i,j], A^1[i,2] + A^1[2,j])$$

Find $A^2[1,3]$

Therefore,

$$\begin{aligned} A^2[1,3] &= \min (A^1[1,3], A^1[1,2] + A^1[2,3]) \\ &= \min (11, 4 + 2) \\ &= \min (11, 6) \\ &= 6 \end{aligned}$$

Therefore,

$$A^2[1,3] = 6$$

Find $A^2[3,1]$

We check whether going from 3 to 1 through vertex 2 gives a shorter path.

$$\begin{aligned} A^2[3,1] &= \min (A^1[3,1], A^1[3,2] + A^1[2,1]) \end{aligned}$$



$$\begin{aligned} &= \min (3, 7 + 6) \\ &= \min (3, 13) \\ &= 3 \end{aligned}$$

Therefore,

$$A^2[3,1] = 3$$

Hence,

	1	2	3
1	0	4	6
2	6	0	2
3	3	7	0

Step 4: Construct A^3 Matrix

Now, **vertex 3 is considered as the intermediate vertex.**

Statement:

From A^2 , we construct A^3 by considering vertex 3 as the intermediate vertex. For every pair of vertices, we check whether passing through vertex 3 produces a shorter path.

The formula is:

$$A^3[i,j] = \min (A^2[i,j], A^2[i,3] + A^2[3,j])$$

Find $A^3[1,2]$

Therefore,

$$\begin{aligned} &A^3[1,2] \\ &= \min (A^2[1,2], A^2[1,3] + A^2[3,2]) \\ &= \min (4, 6 + 7) \\ &= \min (4, 13) \\ &= 4 \end{aligned}$$

Therefore,

$$A^3[1,2] = 4$$

Find $A^3[2,1]$

Therefore,



$$\begin{aligned} &A^3[2,1] \\ &= \min (A^2[2,1], A^2[2,3] + A^2[3,1]) \\ &= \min (6, 2 + 3) \\ &= \min (6, 5) \\ &= 5 \end{aligned}$$

Therefore,

$$A^3[2,1] = 5$$

Final A^3 Matrix

$$A^3 = \begin{bmatrix} 0 & 4 & 6 \\ 5 & 0 & 2 \\ 3 & 7 & 0 \end{bmatrix}$$

This is the **final All Pairs Shortest Path matrix**.

Time Complexity

There are three nested loops:

k=1 to n

i=1 to n

j=1 to n

Therefore, the number of operations is approximately:

$$n \times n \times n = n^3$$

Hence, Time Complexity: $O(n^3)$

Space Complexity

The algorithm uses an $n \times n$ distance matrix.

Therefore,

Space Complexity: $O(n^2)$



MODULE – IV

Short Answer Questions

1. What is Greedy Method?

The Greedy Method is an algorithmic approach that builds a solution step-by-step by choosing the best immediate (locally optimal) choice at each stage, with the hope of obtaining a global optimum. Once a choice is made, it is not changed later.

2. Write any two characteristics of Greedy Algorithm.

1. **Greedy Choice Property** – A locally best choice can lead to a globally optimal solution.
2. **Optimal Substructure** – An optimal solution contains optimal solutions to its subproblems.

3. List the applications of Greedy Method.

The major applications are:

1. Knapsack Problem (Fractional Knapsack)
2. Job Sequencing with Deadlines
3. Minimum Cost Spanning Tree
4. Single Source Shortest Paths

4. Define spanning tree.

A spanning tree of a connected graph is a tree that contains all the vertices of the graph and connects them without forming any cycle. The spanning tree obtained from the discovery edges during BFS/DFS traversal.

5. State the Job-Sequencing Deadline Problem.

The Job Sequencing with Deadlines problem is to schedule a set of jobs on a single processor, where each job has a deadline and profit. A job earns its profit only if it is completed within its deadline. The objective is to select and schedule jobs so that the total profit is maximum.

6. What is the importance of Fractional Knapsack Algorithm in our daily life?

The Fractional Knapsack Algorithm is useful when resources can be divided into fractions and we need to maximize profit within a limited capacity.



Examples: selecting goods for transportation, loading a truck, allocating limited storage space, and choosing resources based on **profit per unit weight**. The greedy criterion is maximum **profit/weight (P/W)**.

7. Find an optimal solution to the knapsack instance.

Using the knapsack instance from your notes:

- $N = 3$
- Capacity $M = 20$
- Profits: (25, 24, 15)
- Weights: (18, 15, 10)

For the maximum profit per unit weight criterion:

Item	Profit P_i	Weight W_i	P_i/W_i
1	25	18	1.4
2	24	15	1.6
3	15	10	1.5

Selection order: **Item 2 $\rightarrow$ Item 3 $\rightarrow$ Item 1**

Therefore,

$$x_1 = 0, \quad x_2 = 1, \quad x_3 = \frac{1}{2}$$

Total weight:

$$15 + 10 \left(\frac{1}{2} \right) = 20$$

Total profit:

$$24 + 15 \left(\frac{1}{2} \right) = \boxed{31.5}$$

Optimal solution: $\boxed{\left(0, 1, \frac{1}{2} \right)}$, with maximum profit = 31.5.

8. Define Single Source Shortest Path Problem.

The Single Source Shortest Path (SSSP) problem is the problem of finding the shortest paths from one given source vertex to all other vertices in a weighted graph.

Example: Dijkstra's algorithm is used to find single-source shortest paths.



9. Mention the types of Binary Tree Traversal.

The three types of binary tree traversal are:

1. **Inorder** – Left → Data → Right (**LDR**)
2. **Preorder** – Data → Left → Right (**DLR**)
3. **Postorder** – Left → Right → Data (**LRD**)

10. Define Minimum Spanning Tree.

A Minimum Spanning Tree (MST) is a spanning tree of a connected weighted graph that connects all vertices with minimum total edge weight and contains no cycles.

Examples of algorithms: Kruskal's Algorithm and Prim's Algorithm.



Long Answer Questions

1. Explain the terms feasible solution, optimal solution and objective function with suitable examples.

Ans:

Introduction

In an optimization problem, we need to identify the possible solutions, check whether they satisfy the given constraints, and then select the best solution. Three important terms are Feasible Solution, Objective Function, and Optimal Solution.

1. Feasible Solution

A feasible solution is a solution that satisfies all the given constraints of the problem.

In other words: A solution is feasible if it satisfies both the explicit and implicit constraints of the problem.

Example: Knapsack Problem

Suppose:

- Knapsack capacity = 10 kg
- Item weights = 4 kg, 3 kg and 5 kg

Selection	Total Weight	Feasible?
Item 1 + Item 2	7 kg	Yes
Item 2 + Item 3	8 kg	Yes
Item 1 + Item 3	9 kg	Yes
Item 1 + Item 2 + Item 3	12 kg	No

The first three are feasible solutions because their total weight does not exceed 10 kg.

The last selection is not feasible because $12 > 10$.

Key Point: Feasible Solution → Satisfies all constraints.

2. Objective Function

The objective function is the function used to measure or evaluate a solution and determine what needs to be maximized or minimized.

Examples:

- Maximize profit
- Minimize cost
- Minimize distance
- Minimize time
- Maximize weight/value

Example: Knapsack Problem

If P_i is the profit of item i and X_i indicates how much of the item is selected, the objective is:

$$\begin{aligned} &\text{Maximize } \sum P_i X_i \\ &\text{subject to: } \sum W_i X_i \leq M \end{aligned}$$



where P_i = profit of item i , W_i = weight of item i , X_i = fraction/selection of item i , and M = capacity of the knapsack.

3. Optimal Solution

An optimal solution is the best feasible solution according to the objective function.

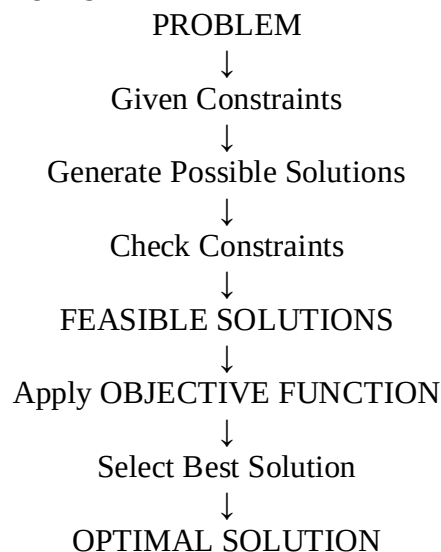
In other words: An optimal solution is a feasible solution that gives the maximum or minimum value of the objective function, depending on the problem.

Example

Solution	Total Profit
S_1	25
S_2	28
S_3	31.5

If the objective is to maximize profit, $S_3 = 31.5$ is the optimal solution because it has the highest profit among the feasible solutions.

Relationship Between the Three Terms



Simple Example – Delivery Company

Constraint: Every delivery location must be visited.

Feasible solution: A route that satisfies all delivery requirements.

Objective function: Minimize total travelling distance.

Optimal solution: The feasible route having the minimum total distance.

Difference Between the Three

Term	Meaning	Example
Feasible Solution	Satisfies all constraints	Knapsack weight $\leq$ capacity
Objective Function	Function used to maximize/minimize a value	Maximize total profit
Optimal Solution	Best feasible solution according to objective	Feasible solution with maximum profit

Conclusion

Feasible Solution → Objective Function → Optimal Solution



2. Find the solution generated by the Job Sequencing with Deadlines problem for N=7 jobs. The profits and deadlines are given in the following table:

Job	J1	J2	J3	J4	J5	J6	J7
Profit	3	5	20	18	1	6	30
Deadline	1	3	4	3	2	1	2

Ans:

Procedure

1. Sort all jobs in decreasing order of profit.
2. Find the maximum deadline.
3. Create slots from 1 to the maximum deadline.
4. Pick jobs one by one and place each job as far as possible from 0, ensuring that it is completed before its deadline.

Given

Job	J1	J2	J3	J4	J5	J6	J7
Profit	3	5	20	18	1	6	30
Deadline	1	3	4	3	2	1	2

Step 1: Arrange jobs in decreasing order of profit

Job	Profit	Deadline
J7	30	2
J3	20	4
J4	18	3
J6	6	1
J2	5	3
J1	3	1
J5	1	2

Maximum deadline = 4

Available time slots

Slot 1	Slot 2	Slot 3	Slot 4
—	—	—	—



Step 2: Schedule the Jobs

J7, deadline 2 → place in Slot 2.

—	J7	—	—
---	----	---	---

J3, deadline 4 → place in Slot 4.

—	J7	—	J3
---	----	---	----

J4, deadline 3 → place in Slot 3.

—	J7	J4	J3
---	----	----	----

J6, deadline 1 → place in Slot 1.

J6	J7	J4	J3
----	----	----	----

Now all four slots are occupied. The remaining jobs J2, J1 and J5 cannot be scheduled because there are no vacant slots satisfying their deadlines.

Gantt Chart

Time	1	2	3	4
Job	J6	J7	J4	J3
Profit	6	30	18	20

J6 → J7 → J4 → J3

Final Scheduled Jobs

Time Slot	Job	Profit	Deadline
1	J6	6	1
2	J7	30	2
3	J4	18	3
4	J3	20	4

Total Profit: Total Profit = $6 + 30 + 18 + 20 = 74$
Optimal Job Sequence = J6 → J7 → J4 → J3
Maximum Profit = 74



3. Consider the below Fractional Knapsack instance with $N=3$, $M=20$. Find the optimal solution using Greedy method.

Item	$P_i(\text{Profit})$	$W_i(\text{Weight})$
I_1	25	18
I_2	24	15
I_3	15	10

Ans:

Given Number of items: $N = 3$, Knapsack capacity: $M = 20$

Step 1: Calculate Profit/Weight Ratio

$$P_1/W_1 = 25/18 = 1.4$$

$$P_2/W_2 = 24/15 = 1.6$$

$$P_3/W_3 = 15/10 = 1.5$$

Arrange the items in decreasing order of profit/weight ratio:

Item 2 > Item 3 > Item 1

Step 2: Select the Items

Knapsack capacity = 20.

Item 2:

Weight = 15, so take the complete item.

$$x_2 = 1$$

$$\text{Remaining capacity} = 20 - 15 = 5$$

Item 3:

Weight = 10, but only 5 units of capacity remain.

$$x_3 = 5/10 = 1/2$$

$$\text{Remaining capacity} = 5 - 5 = 0$$

Item 1:

No capacity remains.

$$x_1 = 0$$

Therefore, $x = (0, 1, 1/2)$.

Step 3: Calculate Total Profit

$$P = 25(0) + 24(1) + 15(1/2)$$

$$= 0 + 24 + 7.5$$

$$= 31.5$$

Therefore, Total Profit = 31.5.

Step 4: Calculate Total Weight

$$W = 18(0) + 15(1) + 10(1/2)$$

$$= 0 + 15 + 5$$

$$= 20$$

Therefore, Total Weight = 20.



Selection Table

Order	Item	Pi/Wi	Fraction xi	Weight Used	Profit
1	Item 2	1.6	1	15	24
2	Item 3	1.5	1/2	5	7.5
3	Item 1	1.4	0	0	0
Total				20	31.5

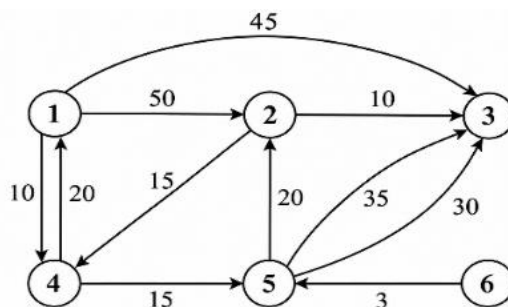
Final Answer

$$x_1 = 0, x_2 = 1, x_3 = 1/2$$

Total Weight = 20, Maximum Profit = 31.5

Hence, the optimal solution is $x = (0, 1, 1/2)$ with a maximum profit of 31.5.

4. Consider the following weighted graph . Find the Single Source Shortest Paths from the given source vertex using the Matrix Method (Dijkstra's Algorithm), and determine the shortest distance from the source vertex to each of the remaining vertices.



Given graph: Source vertex $V_0 = 1$

Represent graph as Cost Matrix

From / To	1	2	3	4	5	6
1	∞	50	45	10	∞	∞
2	∞	∞	10	15	∞	∞
3	∞	∞	∞	∞	30	∞
4	20	∞	∞	∞	15	∞
5	∞	20	35	∞	∞	3
6	∞	∞	∞	∞	∞	∞

means *no direct edge exists*.

∞



Step 1: Initialization

From the cost matrix:

Direct edges from vertex 1 are:

- $1 \rightarrow 2 = 50$
- $1 \rightarrow 3 = 45$
- $1 \rightarrow 4 = 10$
- Others are ∞ (no direct path).

So, the initial distance table is:

Vertex	1	2	3	4	5	6
d[i]	0	50	45	10	∞	∞

Set of vertices included in shortest path, $S = \{1\}$

Step 2: Pick Minimum Distance Vertex (not in S)

Among $\{2(50), 3(45), 4(10)\}$, the **smallest is 10 (vertex 4)**.

So add vertex $4 \rightarrow S = \{1, 4\}$.

Now, **update the distances** of vertices that are directly connected to vertex 4.

From cost matrix,

$4 \rightarrow 5 = 15$ and $4 \rightarrow 1 = 20$ (already in S).

So check via vertex 4:

- Distance($1 \rightarrow 4 \rightarrow 5$) = $10 + 15 = 25$
Compare with existing $d[5]$ (∞) $\rightarrow$ update to 25.

Updated table:

Step	Vertices Selected (S)	1	2	3	4	5	6
1	{1}	-	50	45	10	∞	∞
2	{1,4}	-	50	45	10	25	∞

Step 3: Pick next minimum vertex

Now check distances $\{2(50), 3(45), 5(25)\} \rightarrow$ **smallest is 25 (vertex 5)**

So, add vertex $5 \rightarrow S = \{1, 4, 5\}$

Now update distances using edges from 5:

- $5 \rightarrow 2 = 20 \Rightarrow$ via $1 \rightarrow 4 \rightarrow 5 \rightarrow 2 = 10 + 15 + 20 = 45$



- $5 \rightarrow 3 = 35 \Rightarrow \text{via } 1 \rightarrow 4 \rightarrow 5 \rightarrow 3 = 10 + 15 + 35 = 60$
- $5 \rightarrow 6 = 3 \Rightarrow \text{via } 1 \rightarrow 4 \rightarrow 5 \rightarrow 6 = 10 + 15 + 3 = 28$

Compare and update:

- $d[2] = \min(50, 45) = 45$
- $d[3] = \min(45, 60) = 45$ (no change)
- $d[6] = 28$ (new)

Updated table:

Step	Vertices Selected (S)	1	2	3	4	5	6
3	{1,4,5}	-	45	45	10	25	28

Step 4: Next minimum $\rightarrow$ vertex 6 (28)

Add vertex 6 $\rightarrow S = \{1,4,5,6\}$

Vertex 6 has no outgoing edges (∞ in matrix), so no updates.

Table remains same.

Step	Vertices Selected (S)	1	2	3	4	5	6
4	{1,4,5,6}	-	45	45	10	25	28

Step 5: Next minimum $\rightarrow$ vertex 2 (45)

Add vertex 2 $\rightarrow S = \{1,4,5,6,2\}$

Update using edges from 2:

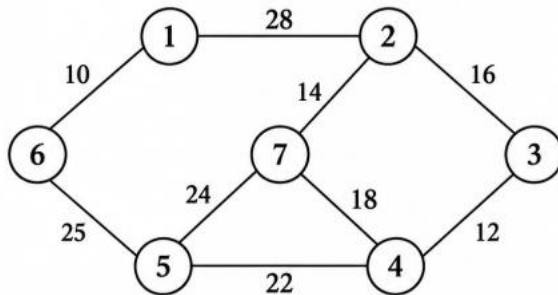
$2 \rightarrow 3 = 10 \Rightarrow \text{via } 1 \rightarrow 4 \rightarrow 5 \rightarrow 2 \rightarrow 3 = 10 + 15 + 20 + 10 = 55$ (no improvement since $45 < 55$)

Final Table:

Step	Vertices Selected (S)	1	2	3	4	5	6
1	{1}	-	50	45	10	∞	∞
2	{1,4}	-	50	45	10	25	∞
3	{1,4,5}	-	45	45	10	25	∞
4	{1,4,5,2}	-	45	45	10	25	∞
5	{1,4,5,2,3}	-	45	45	10	25	∞



5. Write and explain Prim's algorithm for finding the Minimum Cost Spanning Tree of a connected graph. Execute the algorithm on the given graph.



Prim's Algorithm

- Prim's Algorithm is a famous greedy algorithm.
- It is used for finding the Minimum Spanning Tree (MST) of a given graph.
- To apply Prim's Algorithm, the given graph must be weighted, connected and undirected.

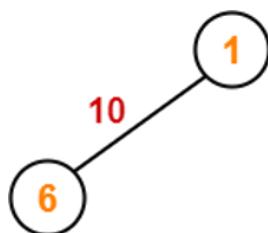
Prim's Algorithm Implementation-

The implementation of Prim's Algorithm is explained in the following steps-

1. Randomly choose any vertex. The vertex connecting to the edge having least weight is usually selected.
2. Find all the edges that connect the tree to new vertices. Find the least weight edge among those edges and include it in the existing tree. If including that edge creates a cycle, then reject that edge and look for the next least weight edge.
3. Keep repeating step-02 until all the vertices are included and Minimum Spanning Tree (MST) is obtained.
4. If there are n vertices in the graph, then each spanning tree has n vertices and $n - 1$ edges.

Solution: This graph contains 7 vertices and thus we obtain a spanning tree with 6 edges

Step 1: The starting node is 6. The possible edges from 6 that can be added in the minimum spanning tree 'S' are (6, 1), (6, 5). The costs of these edges are 10 and 25. Since, the edge (6, 1) has the lowest cost among the possible edges. It is included in the tree 'S'.

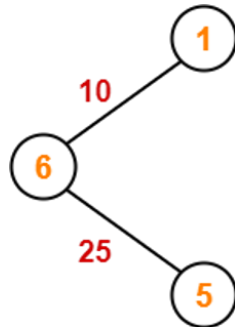


Total Weight = 10

Step 2: The nodes included in tree S are 6 and 1 . The possible edges from these nodes that can be added in the minimum spanning tree 'S' are (1, 2), (6, 5). The costs of these edges are 28 and 25.

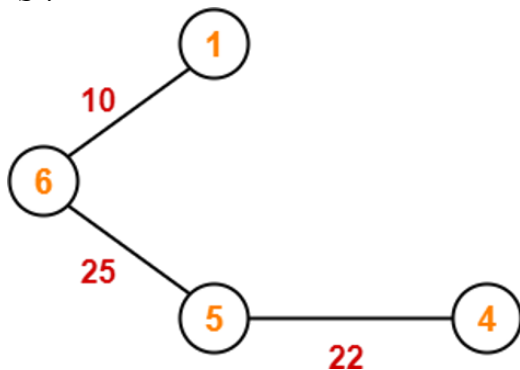


Since, the edge (6, 5) has the lowest cost among the possible edges. It is included in the tree 'S'.



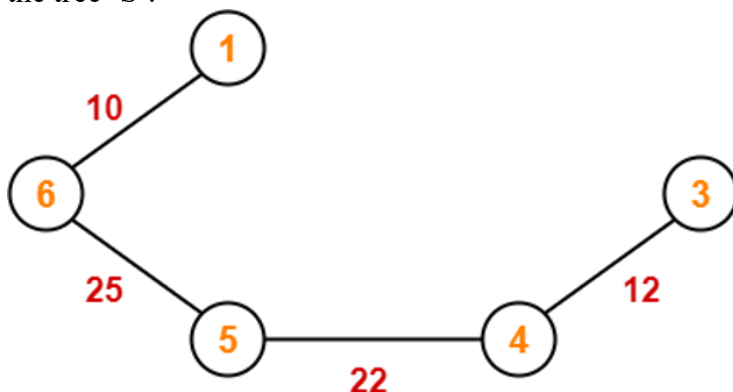
Total Weight = $10 + 25 = 35$

Step 3: The nodes included in tree S are 1, 6 and 5 . The possible edges from these nodes that can be added in the minimum spanning tree 'S' are (1, 2), (5, 4) (5, 7). The costs of these edges are 28 , 22 and 24. Since, the edge (5, 4) has the lowest cost among the possible edges. It is included in the tree 'S'.



Total Weight = $10 + 25 + 22 = 57$

Step 4: The nodes included in tree S are 1,6,5 and 4 . The possible edges from these nodes that can be added in the minimum spanning tree 'S' are (1, 2), (5, 7), (4, 3) (4, 7). The costs of these edges are 28 , 24, 12 and 18. Since, the edge (4, 3) has the lowest cost among the possible edges. It is included in the tree 'S'.

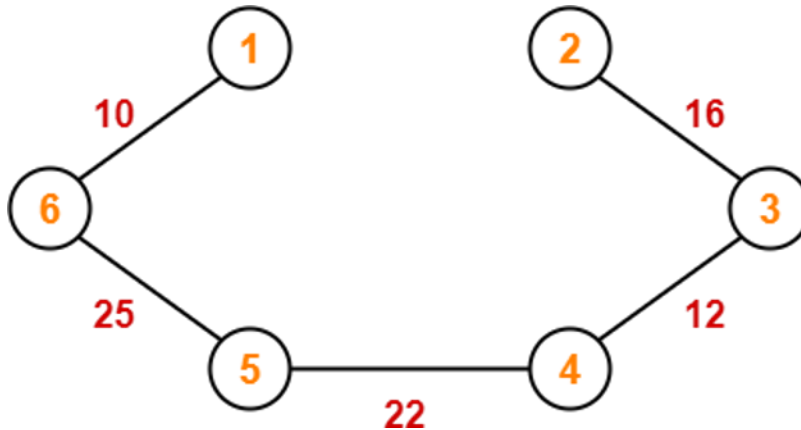




Total Weight = $10 + 25 + 22 + 12 = 69$

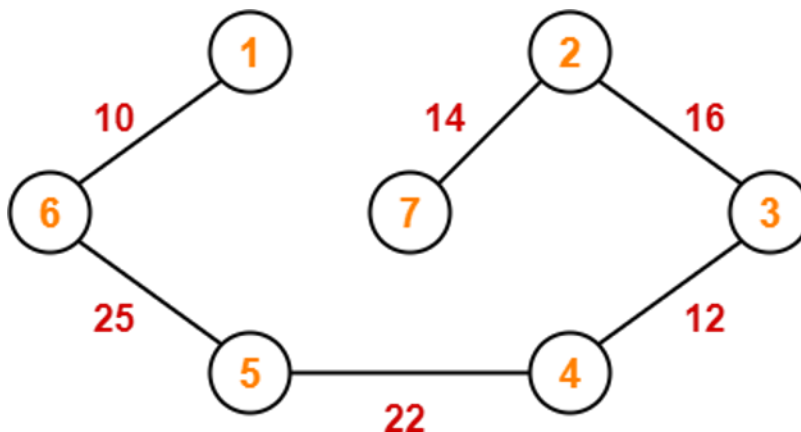
Step 5: The nodes included in tree S are 1,6,5,4 and 3 . The possible edges from these nodes that can be added in the minimum spanning tree 'S' are (1, 2), (5, 7) (4, 7) (3, 2). The costs of these edges are 28 24, 18 and 16. Since, the edge (3, 2) has the lowest cost among the possible edges. It is included

in the tree 'S'.



Total Weight = $10 + 25 + 22 + 12 + 16 = 85$

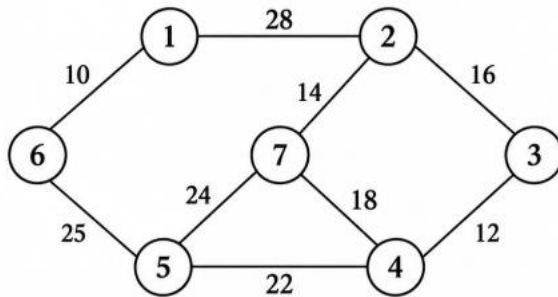
Step 6: The nodes included in tree S are 1,6,5,4,3 and 2 . The possible edges from these nodes that can be added in the minimum spanning tree 'S' are (1, 2), (5, 7) (4, 7) (2, 7). The costs of these edges are 28, 24, 18 and 14. Since, the edge (2, 7) has the lowest cost among the possible edges. It is included in the tree 'S'.



Total Weight = $10 + 25 + 22 + 12 + 16 + 14 = 99$



6. Generate the Minimum Cost Spanning Tree (MCST) for the given graph by applying Kruskal's algorithm.



Ans:

- Kruskal's Algorithm is a famous greedy algorithm.
- It is used for finding the Minimum Spanning Tree (MST) of a given graph.
- To apply Kruskal's algorithm, the given graph must be weighted, connected and undirected.

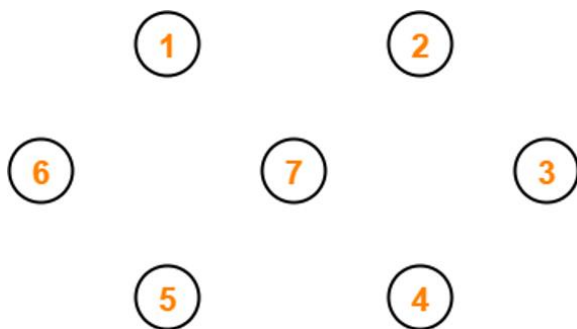
Kruskal's Algorithm Implementation-

The implementation of Kruskal's Algorithm is explained in the following steps-

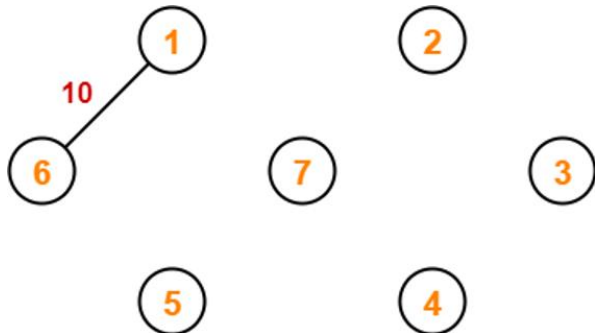
1. Sort all the edges in decreasing order of their weights (low weight to high weight).
2. Take the edge with the lowest weight and use it to connect the vertices of graph. If adding an edge creates a cycle, then reject that edge and go for the next least weight edge.
3. Keep adding edges until all the vertices are connected and a Minimum Spanning Tree (MST) is obtained.
4. If there are n vertices in the graph, then each spanning tree has n vertices and $n - 1$ edges.

Solution:

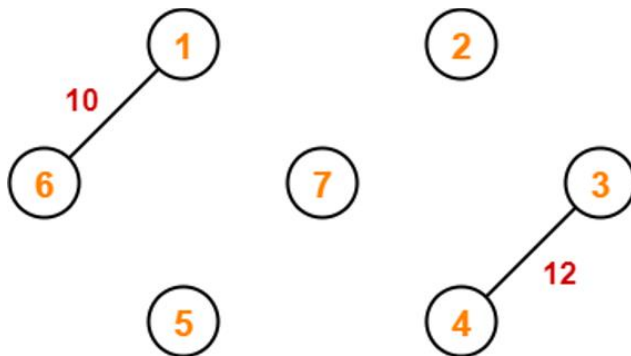
Step 1:



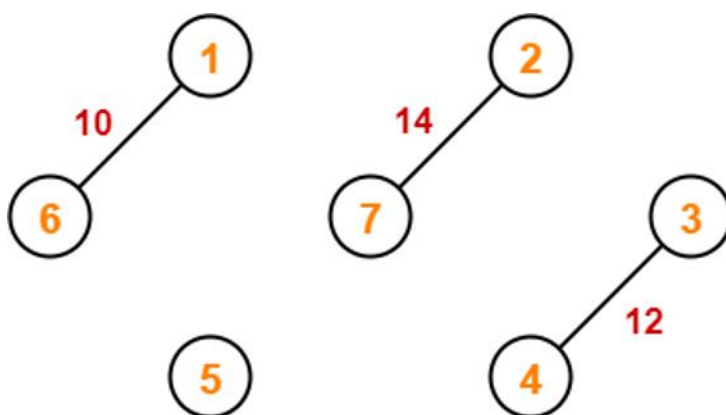
Step 2: The first edge in the sorted list i.e, (1,6) that has the least cost is included in the spanning tree. Then the edges 1 and 6 are merged as single set.



Step 3: The next edge in the sorted list i.e, (3,4) that has the least cost is included in the spanning tree. Then the edges 3 and 4 are merged as single set.



Step 4: The next edge in the sorted list i.e, (2,7) that has the least cost is included in the spanning tree. Then the edges 2 and 7 are merged as single set.

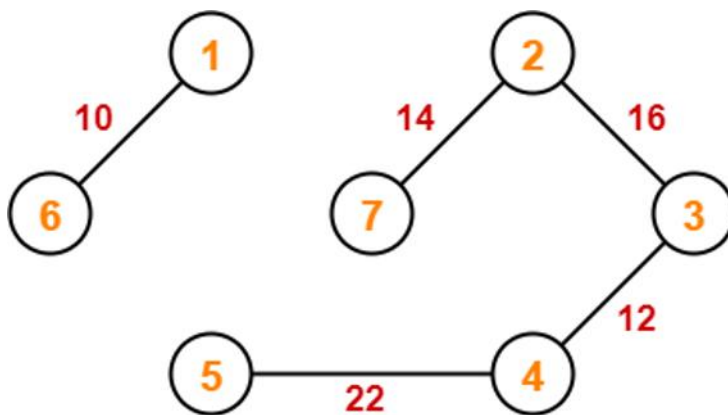


Step 5: The next edge in the sorted list i.e, (2,3) that has the least cost is included in the spanning tree. Then the edges 2 and 3 are merged as single set

Step 6: The next edge in the sorted list i.e, (4,7) that has the least cost is included in the spanning tree. An edge creates a cycle, and then rejects that edge and go for the next least weight edge.

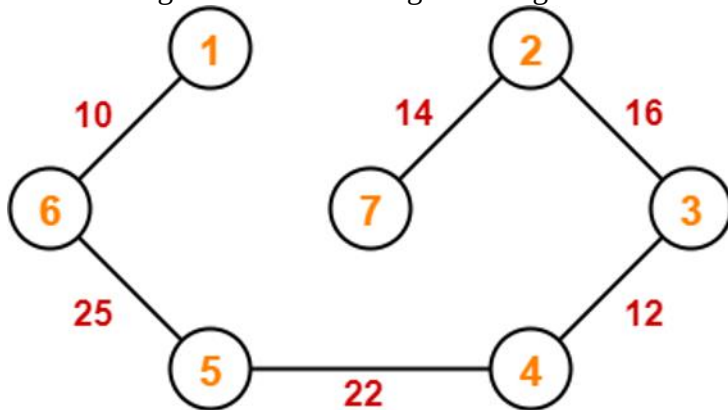


Step 7: The next edge in the sorted list i.e, (4,5) that has the least cost is included in the spanning tree. Then the edges 4 and 5 are merged as single set.



Step 8: The next edge in the sorted list i.e, (5,7) that has the least cost is included in the spanning tree. An edge creates a cycle, and then rejects that edge and go for the next least weight edge.

Step 9: The next edge in the sorted list i.e, (5,6) that has the least cost is included in the spanning tree. Then the edges 5 and 6 are merged as single set.



Total weight = 10 + 12 + 14 + 16 + 22 + 25 = 99



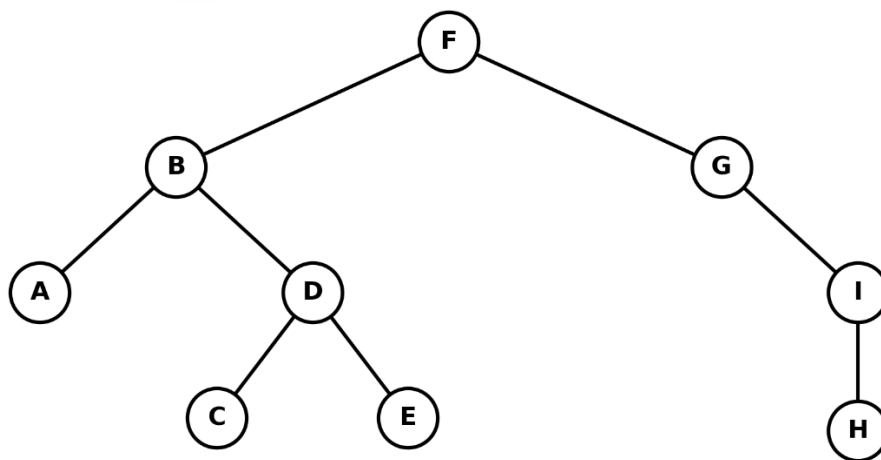
7. Explain the Various Tree Traversal Techniques for Binary Trees with Suitable Examples. Binary Tree

s

A binary tree is a finite set of nodes which is either empty or consists of a root and two disjoint binary trees called the left subtree and right subtree. In a traversal, every element of the binary tree is visited exactly once.

The three traversal techniques are Inorder, Preorder and Postorder.

Binary Tree Traversal Example



Inorder: A-B-C-D-E-F-G-H-I
Preorder: F-B-A-D-C-E-G-I-H
Postorder: A-C-E-D-B-H-I-G-F

Inorder Traversal

In inorder traversal, the nodes are visited in the order L-D-R, where L means moving left, D means displaying the data, and R means moving right.

Algorithm inorder(x):

1. If $x \neq \text{NULL}$
2. Then inorder(left(x));
3. Output key(x);
4. Inorder(right(x));

Example: Inorder = A-B-C-D-E-F-G-H-I.

Preorder Traversal

In preorder traversal, the nodes are visited in the order D-L-R.

Algorithm preorder(x):

1. If $x \neq \text{NULL}$
2. Then output key(x);
3. Preorder(left(x));
4. Preorder(right(x));

Example: Preorder = F-B-A-D-C-E-G-I-H.



Postorder Traversal

In postorder traversal, the nodes are visited in the order L-R-D.

Algorithm postorder(x):

1. If $x \neq \text{NULL}$
2. Then Postorder(left(x));
3. Postorder(right(x));
4. Output key(x);

Example: Postorder = A-C-E-D-B-H-I-G-F. **Difference Between Traversals**

Traversal	Order	Main Idea
Inorder	L-D-R	Left $\rightarrow$ Root $\rightarrow$ Right
Preorder	D-L-R	Root $\rightarrow$ Left $\rightarrow$ Right
Postorder	L-R-D	Left $\rightarrow$ Right $\rightarrow$ Root

Conclusion: Binary tree traversal visits every node exactly once. The three techniques are Inorder, Preorder and Postorder.

7. Differentiate Between Breadth First Search (BFS) and Depth First Search (DFS) with Suitable Examples.

1. Graph Traversal

The process of traversing all the nodes or vertices of a graph is called graph traversal. The two graph traversal techniques are Depth First Search (DFS) and Breadth First Search (BFS).

2. Depth First Search (DFS)

DFS explores each possible path to its conclusion before another path is tried. It goes as far as possible and then backtracks to try another path.

Steps of DFS

1. Select an unvisited node v , visit it and treat it as the current node.
2. Find an unvisited neighbour of the current node, visit it and make it the new current node.
3. If the current node has no unvisited neighbours, backtrack to its parent.
4. Repeat until no more nodes can be visited.
5. Repeat for remaining unvisited nodes.

Implementation of DFS:

```
DFS(u)
{
    Mark u as visiting
    For each vertex v directly reachable from u
        If v is unvisited
            DFS(v)
}
```

Properties: DFS visits all vertices and edges in the connected component of v . The discovery edges labeled by DFS form a spanning tree of the connected component.



3. Breadth First Search (BFS)

BFS is a graph traversal method in which each node on the same level is checked before the search proceeds to the next level. BFS uses a queue to store visited vertices.

Steps of BFS

1. Mark all vertices as unvisited.
2. Choose a vertex v , mark it as visited and put it at the end of the queue.
3. For each vertex on the list, examine all vertices adjacent to v .
4. Mark unvisited adjacent vertices and put them at the rear of the queue.
5. Remove a vertex from the front of the queue and repeat.
6. Continue until the queue is empty.

Implementation of BFS:

While queue Q is not empty

 Deque the first vertex u

 For each vertex v directly reachable from u

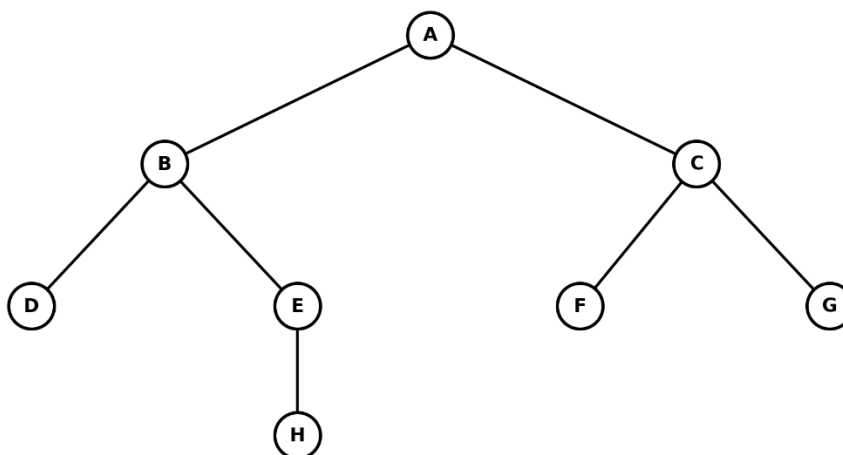
 If v is unvisited

 Enqueue v to Q

 Mark v as visited

Example BFS: A-B-C-D-E-F-G-H.

Q8 - BFS vs DFS (Starting from A)



BFS: A-B-C-D-E-F-G-H (level by level)

DFS: A-B-D-E-H-C-F-G (depth-wise/backtracking)

Difference Between BFS and DFS

Feature	BFS	DFS
Approach	Level by level	Depth-wise
Data structure	Queue	Recursion / backtracking



Exploration	Same-level vertices first	Goes as deep as possible
Backtracking	Not the main approach	Main feature
Spanning tree	BFS spanning tree	DFS spanning tree

Conclusion: BFS explores a graph level by level using a queue, whereas DFS explores depth-wise using recursion/backtracking.

8. Explain Connected Components and Bi-connected Components in Graphs with Suitable Examples and Algorithms.

1. Connected Component

If G is a connected undirected graph, all the vertices can be visited in the first call to BFS. The subgraph obtained after traversing the graph using BFS represents the connected component. BFS can therefore be used to determine whether G is connected.

Example: If vertices A, B, C, D and E can be reached from A , then $\{A, B, C, D, E\}$ forms a connected component. A separate group of unreachable vertices forms another connected component.

Algorithm Using BFS

Connected-Component(G)

1. Mark all vertices as unvisited.
2. Select an unvisited vertex v .
3. Apply BFS(v).
4. All vertices visited by this BFS form one connected component.
5. If unvisited vertices remain, select another vertex.
6. Repeat until all vertices are visited.

2. Bi-connected Component

A connected undirected graph is said to be bi-connected if it remains connected after removal of any one vertex and the edges that are incident upon that vertex.

3. Articulation Point

An articulation point of a connected undirected graph is a vertex whose removal disconnects the graph. It is also known as a cut point.

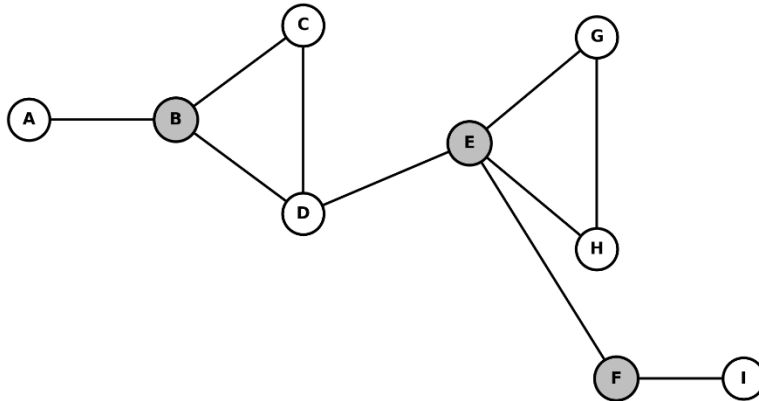
Example: In the given example, B, E and F are articulation points. Removing B and its incident edges divides the graph into two components. Similarly, removing E or F disconnects the graph.

4. Bi-connected Graph

A graph G is said to be bi-connected if it contains no articulation point.



Connected / Bi-connected Components



B, E and F are articulation (cut) points in this illustrative graph.
Removing an articulation point disconnects the graph.

5. Algorithm to Construct a Bi-connected Graph

1. For each articulation point a do.
2. Let $B_1, B_2, \dots, B_k$ be the bi-connected components containing a .
3. Let $V_i \in B_i, V_i \neq a$.
4. Add (V_i, V_{i+1}) to graph G .

Here, a is the articulation point, B_i is a bi-connected component and V_i is a vertex belonging to B_i .

Importance: If an articulation point exists in a communication network, failure of that joint node can disconnect communication between other stations.

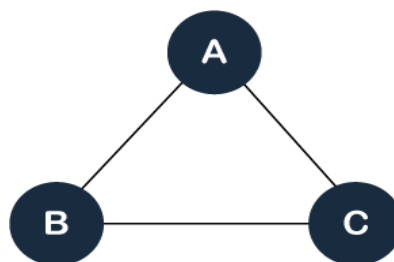
Conclusion: A connected component identifies a group of connected vertices, while a bi-connected graph remains connected after removal of any one vertex.

10. Define a Spanning Tree. Explain Its Properties and Discuss the Methods for Finding a Spanning Tree of a Connected Graph with a Suitable Example.

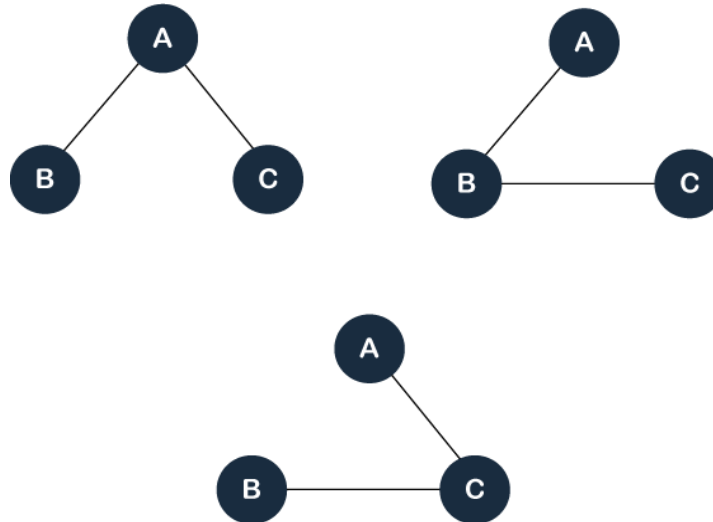
1. Definition of Spanning Tree

A spanning tree of a connected graph is formed by the discovery edges obtained during graph traversal. For a connected graph, the set of discovery edges obtained by BFS gives a spanning tree. A depth first search spanning tree can be obtained similarly.

Example:



Solution: The possible spanning trees are



Properties of Spanning Tree

1. A spanning tree contains all the vertices of the connected graph.
2. A spanning tree is connected.
3. For V vertices, a spanning tree contains $V - 1$ edges.
4. A spanning tree contains no cycles.
5. Removing an edge from a spanning tree disconnects the tree.
6. Adding an additional edge to a spanning tree creates a cycle.
7. BFS can be used to obtain a BFS spanning tree.
8. DFS can be used to obtain a DFS spanning tree.

Methods for Finding a Spanning Tree

Method 1: BFS Spanning Tree

BFS visits vertices level by level. The discovery edges obtained during BFS form the BFS spanning tree of a connected graph.

Method 2: DFS Spanning Tree

DFS explores one path as deeply as possible before backtracking. The discovery edges obtained during DFS form the DFS spanning tree of the connected component.

Suitable Example

Consider a connected graph with vertices A, B, C, D, E and F. Starting from A, BFS visits vertices level by level. For example, the traversal may be A-B-C-D-E-F. The discovery edges form the BFS spanning tree.

Using DFS, the traversal may be A-B-D-E-C-F. The discovery edges form the DFS spanning tree.



Comparison

Feature	BFS Spanning Tree	DFS Spanning Tree
Traversal	Level by level	Depth-wise
Main structure	Queue	Recursion / backtracking
Tree obtained from	BFS discovery edges	DFS discovery edges
Exploration	Same-level vertices first	Deep path first
Backtracking	Not the main approach	Yes

Conclusion: For a connected graph, both BFS and DFS can be used to obtain a spanning tree. The resulting trees may be different because BFS explores level by level while DFS explores depth-wise.

MODULE – V

Short Answer Questions

1. What is Branch and Bound?

Answer: Branch and Bound is a general algorithmic technique used for solving optimization problems. It systematically explores the solution space by branching into subproblems and uses bounds to eliminate subproblems that cannot produce a better solution.

2. What is the Travelling Salesperson Problem (TSP)?

Answer: TSP is the problem of finding the minimum-cost tour that visits every city exactly once and returns to the starting city. It is an important application of Branch and Bound.

3. What is the 0/1 Knapsack Problem?

Answer: The 0/1 Knapsack problem is to select items with given weights and profits so that the total weight does not exceed the knapsack capacity and the maximum profit is obtained. Each item can either be selected (1) or rejected (0).

4. What is LC Branch and Bound?

Answer: LC (Least Cost) Branch and Bound selects the next live node having the least cost or bound value for expansion. A priority queue is generally used to select the most promising node.

5. What is FIFO Branch and Bound?

Answer: FIFO (First-In-First-Out) Branch and Bound selects live nodes in the order in which they are generated. It uses a queue for storing live nodes.

6. What is an NP-Hard problem?

Answer: An NP-Hard problem is a problem that is very difficult to solve efficiently. If any NP-Hard problem can be solved in polynomial time, then all NP problems can also be solved in polynomial time.

Example: Travelling Salesperson Problem (optimization version), 0/1 Knapsack (optimization version).



7. What is an NP-Complete problem?

Answer: A problem is NP-Complete if it belongs to NP and is NP-Hard. Examples include SAT, 3-SAT, and Hamiltonian Cycle.

8. What is a non-deterministic algorithm?

Answer: A non-deterministic algorithm can make a choice among multiple possible alternatives and is considered successful if at least one sequence of choices leads to a valid solution.

9. What is Cook's Theorem?

Answer: Cook's Theorem states that the Boolean Satisfiability Problem (SAT) is NP-Complete. It was the first problem proven to be NP-Complete.

10. Differentiate between NP-Hard and NP-Complete problems.

Answer: NP-Hard problems need not belong to NP, whereas NP-Complete problems must belong to NP and also be NP-Hard. NP-Hard problems may not have solutions verifiable in polynomial time, while solutions to NP-Complete problems can be verified in polynomial time. Examples: optimization version of TSP is NP-Hard; SAT and 3-SAT are NP-Complete.

Long Answer Questions

1.Explain the Branch and Bound technique in detail. Discuss its general method, basic terminology, state-space tree, branching, bounding function, and pruning with a suitable example.

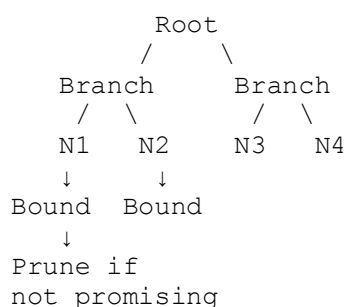
Branch and Bound is a problem-solving technique used to solve optimization problems. It systematically explores possible solutions using a state-space tree and eliminates branches that cannot give a better solution.

Examples: 0/1 Knapsack, Travelling Salesperson Problem (TSP).

General Method

The basic steps are:

1. Start with the **root node**, representing the complete problem.
2. **Branch** the problem into smaller subproblems.
3. Calculate a **bound** for each subproblem.
4. Compare the bound with the best solution found so far.
5. **Prune** the subproblems that cannot produce a better solution.
6. Continue until the optimal solution is obtained.





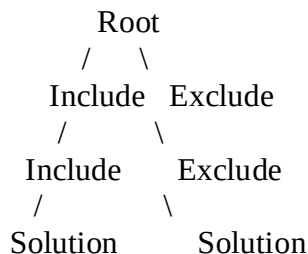
Terminology

Term	Meaning
State-space tree	Tree representing all possible solutions
Node	A partial solution
Branching	Dividing a problem into subproblems
Bound	Estimated best possible solution from a node
Live node	Node that may still be explored
Dead node	Node that is discarded or completely explored
Pruning	Removing a node that cannot give a better solution

State-Space Tree

A **state-space tree** represents all possible choices for a problem. For example, in **0/1 Knapsack**, each item has two choices:

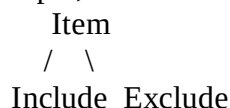
- **Include** the item $\rightarrow 1$
- **Exclude** the item $\rightarrow 0$



Thus, the tree represents different combinations of items.

Branching

Branching means dividing a problem into smaller choices. For example, for an item in 0/1 Knapsack:



Each branch represents one possible decision.

Bounding Function

A bounding function calculates the best possible value that can be obtained from a particular node. For a maximization problem:

If the bound of a node is less than or equal to the current best solution, that node can be discarded. For example:

Current best = 50



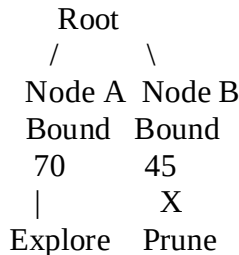
Node A $\rightarrow$ Bound = 70 $\rightarrow$ Explore

Node B $\rightarrow$ Bound = 45 $\rightarrow$ Prune

Node B is removed because it cannot produce a solution better than 50.

Pruning

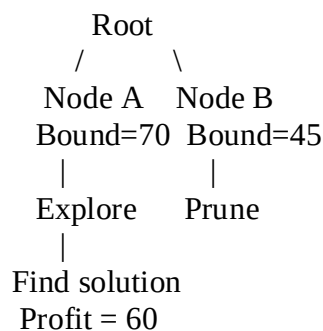
Pruning means eliminating unpromising branches from the state-space tree.



Since Node B has a bound of 45 and the current best solution is 50, Node B is pruned.

Suitable Example – 0/1 Knapsack

Suppose the current best profit is 50.



After finding a solution with profit 60, any node whose bound is 60 or less can be pruned.

Thus, **Branch and Bound** finds an optimal solution by **branching** into possible solutions, calculating a **bound**, and **pruning** branches that cannot improve the current best solution. This reduces unnecessary searching and makes optimization problems easier to solve.

2. Define the term branch and bound. What are the methods of branch and bound?

Branch and Bound is a technique used to solve optimization problems by dividing the problem into smaller subproblems and finding the best solution.

It uses:

- Branching – divides the problem into smaller choices.
- Bounding – calculates the best possible solution from each choice.
- Pruning – removes choices that cannot give a better solution.

Examples: 0/1 Knapsack and Travelling Salesperson Problem (TSP).



2. Methods of Branch and Bound

The main methods are:

1. FIFO Branch and Bound

- FIFO means First In First Out.
- Live nodes are stored in a queue.
- The node that is generated first is explored first.
- It works similar to Breadth-First Search (BFS).

Root $\rightarrow$ A $\rightarrow$ B $\rightarrow$ C $\rightarrow$ D

↑
Explore in order

Data structure used: Queue

2. LIFO Branch and Bound

- LIFO means Last In First Out.
- Live nodes are stored in a stack.
- The most recently generated node is explored first.
- It works similar to Depth-First Search (DFS).

Root
|
A
|
B $\leftarrow$ Explore first

Data structure used: Stack

3. Least-Cost (LC) Branch and Bound

- The node with the least cost (best bound) is selected first.
- A priority queue is used.
- It always selects the most promising live node.

Root
/ \
Cost=20 Cost=10
↑
Selected first

Data structure used: Priority Queue

3. Comparison

Method	Selection of Node	Data Structure
--------	-------------------	----------------



FIFO	First generated node	Queue
LIFO	Last generated node	Stack
LC	Least-cost/best-bound node	Priority Queue

The three important methods of Branch and Bound are FIFO, LIFO, and Least-Cost Branch and Bound. All use branching, bounding, and pruning to find an optimal solution efficiently.

3. Explain the basic concepts of P, NP, NP-Hard, and NP-Complete classes. Discuss the relationship between these classes with suitable examples.

1. P Class

P (Polynomial Time) contains problems that can be solved in polynomial time by a computer.
Example:

- Binary Search
- Merge Sort
- Shortest Path Problem

Problems that can be solved efficiently are called P Class Problems

2. NP Class

NP (Nondeterministic Polynomial Time) contains problems whose solutions can be verified in polynomial time.

Example:

- Hamiltonian Cycle
- Subset Sum

Ex: If someone gives us a solution, we can check it quickly.

3. NP-Hard Class

An NP-Hard problem is a problem that is at least as difficult as every problem in NP.
It may or may not belong to NP.

Examples:

- TSP optimization problem
- 0/1 Knapsack optimization problem

These are very difficult optimization problems.

4. NP-Complete Class

A problem is NP-Complete if it satisfies both conditions:

1. It belongs to NP.
2. It is NP-Hard.



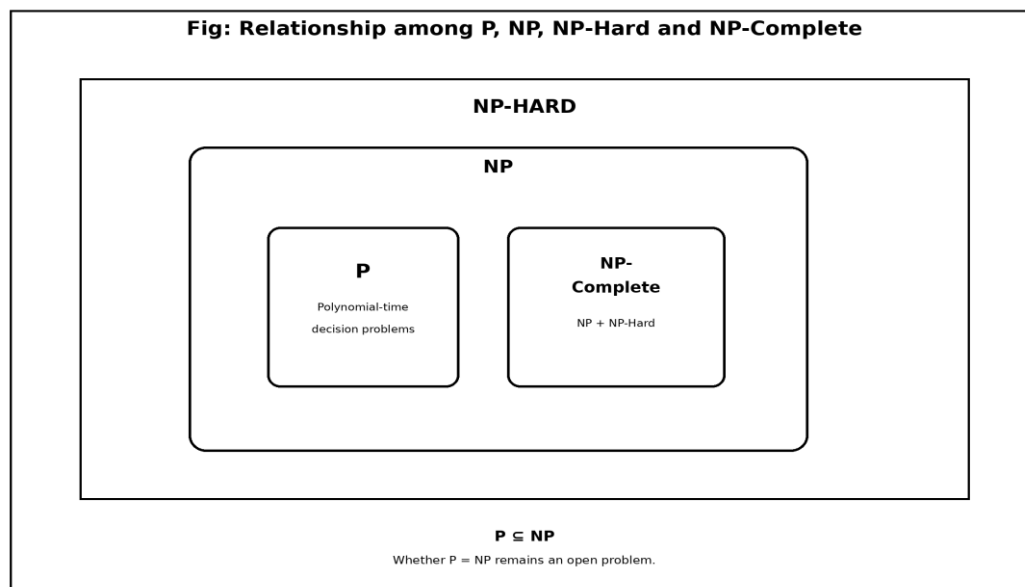
Examples:

- Hamiltonian Cycle
- Subset Sum
- 0/1 Knapsack decision problem
- TSP decision problem

NP-Complete problems are problems that are both in NP and NP-Hard.

5. Relationship Between the Classes

The relationship can be represented as:



- $P \subseteq NP$
- $NP\text{-Complete} \subseteq NP$
- NP-Complete problems are also NP-Hard.
- NP-Hard can contain problems that are not in NP.

Class	Main Idea	Example
P	Can be solved quickly	Binary Search
NP	Solution can be verified quickly	Hamiltonian Cycle
NP-Hard	At least as hard as NP problems	TSP optimization
NP-Complete	Both NP and NP-Hard	TSP decision problem

$P \subseteq NP$, and NP-Complete problems are the intersection of NP and NP-Hard problems.



4. What are non-deterministic algorithms? Compare deterministic and non-deterministic algorithms

A non-deterministic algorithm is an algorithm in which, at some step, more than one possible choice can be made.

It can be thought of as guessing a possible solution and then checking whether the solution is correct. Example: In the Hamiltonian Cycle problem, the algorithm can guess a possible path and then verify whether it forms a Hamiltonian cycle.

Deterministic Algorithm

A **deterministic algorithm** always follows **one fixed sequence of steps** for a given input. For the same input, it always produces the same result in the same way.

Example: Binary Search, Merge Sort.

Deterministic Algorithm

Has only **one possible choice** at each step.

Follows a fixed sequence of steps.

Same input follows the same execution path.

Easy to implement on a conventional computer.

Example: Binary Search

Example

Suppose we need to find whether a path exists between two vertices.

Deterministic:

Choose one path



Check the path



Continue systematically

Non-Deterministic Algorithm

May have **multiple possible choices** at a step.

Can be viewed as choosing or guessing a solution.

Different possible execution paths may exist.

Mainly a theoretical concept used in complexity analysis.

Example: Hamiltonian Cycle

Non-Deterministic:

Start



A B C



Try possible paths



Check the solution



5. Compare LC Branch and Bound and FIFO Branch and Bound.

FIFO Branch and Bound

FIFO Branch and Bound uses the First-In-First-Out principle. Live nodes are stored in a queue, and the first generated live node is selected as the next E-node.

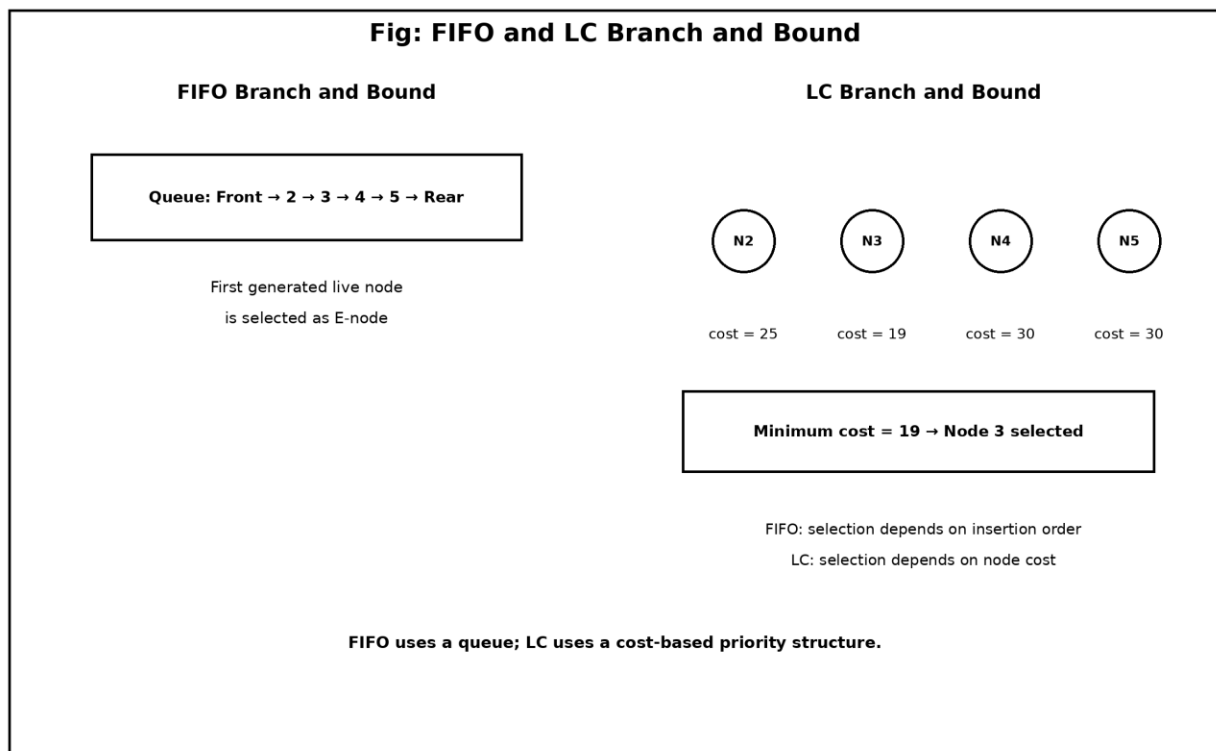
LC Branch and Bound

LC Branch and Bound means Least Cost Branch and Bound. A cost is associated with each node, and the live node with the least cost is selected for expansion.

Example

Suppose the costs of the live nodes are Node 2 = 25, Node 3 = 19, Node 4 = 30 and Node 5 = 30. In LC Branch and Bound, Node 3 is selected because it has the minimum cost.

FIFO Branch and Bound	LC Branch and Bound
First-In-First-Out	Least Cost
Uses a queue	Uses cost-based priority structure
First live node is selected	Least-cost live node is selected
Breadth-first type	Cost-directed
Selection depends on insertion order	Selection depends on node cost
Simple selection rule	Requires cost comparison





FIFO selects a node according to its insertion order, while LC selects the most promising live node according to its cost.

6. Explain Cook's Theorem (Cook-Levin Theorem) in detail. Discuss its significance in computational complexity theory.

Cook's Theorem

Cook's Theorem, also called the Cook-Levin Theorem, establishes that the Boolean Satisfiability Problem (SAT) is NP-Complete. Stephen Arthur Cook and L. A. Levin independently established this result in 1971.

Satisfiability Problem

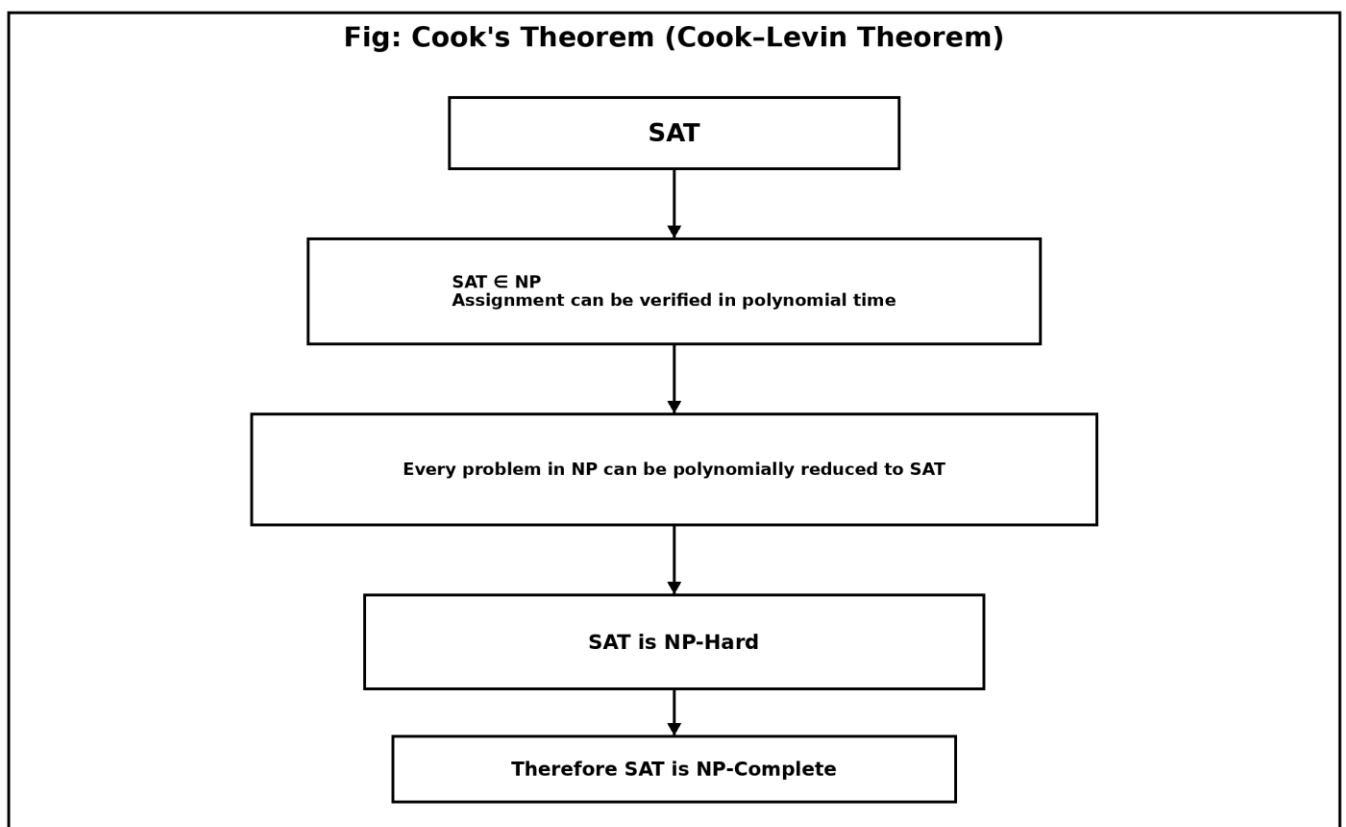
The satisfiability problem determines whether a Boolean formula is true for some assignment of truth values to its variables.

For example, for a Boolean formula $F(x_1, x_2, x_3)$, the question is whether there exists an assignment to x_1, x_2 and x_3 that makes F true.

Main Result of Cook's Theorem

- SAT belongs to NP because a proposed truth assignment can be checked in polynomial time.
- Every problem in NP can be reduced to SAT in polynomial time.
- Therefore SAT is NP-Hard.
- Since SAT belongs to NP and is NP-Hard, SAT is NP-Complete.

Fig: Cook's Theorem (Cook-Levin Theorem)





Significance

- SAT became a fundamental NP-Complete problem.
- It provided the basis for proving many other problems NP-Complete.
- Polynomial-time reduction became a central technique in complexity theory.
- It established a foundation for studying the relationship between P, NP, NP-Hard and NP-Complete.
- Problems such as Hamiltonian Tour, Vertex Cover and Clique were subsequently related to SAT through polynomial reductions.

Importance of Polynomial Reducibility

If a problem can be transformed into another problem in polynomial time, the second problem can be used as a basis for studying the difficulty of the first problem. This idea is central to NP-Completeness proofs.

Cook's Theorem is a fundamental result of computational complexity theory because it established SAT as NP-Complete and provided the foundation for the systematic study of NP-Complete problems.

7. What are NP-hard and NP-Complete Problems? Explain their Characteristics.

NP-Hard Problems

A problem is called NP-Hard if problems in NP can be reduced to it in polynomial time. An NP-Hard problem does not necessarily belong to NP.

Characteristics of NP-Hard Problems

- They are at least as hard as the problems in NP under polynomial-time reduction.
- They need not be decision problems.
- They need not belong to NP.
- They may be optimization problems.
- The optimization version of a problem may be NP-Hard even when its corresponding decision version is NP-Complete.

Example

Knapsack optimization problem is an example of an NP-Hard optimization problem.

NP-Complete Problems

A problem is NP-Complete if it belongs to NP and is also NP-Hard.

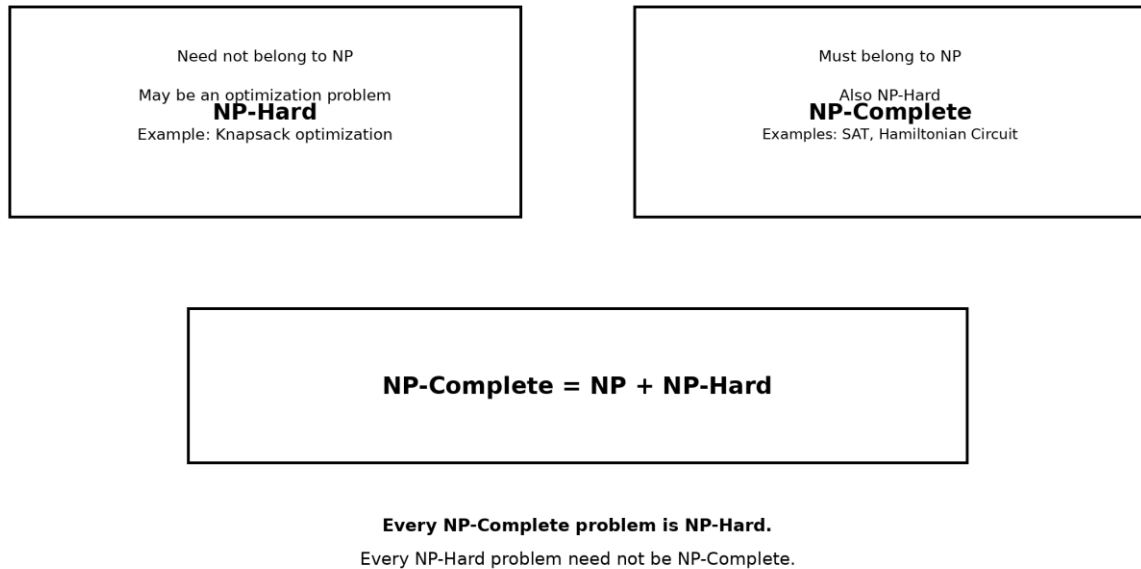
Characteristics of NP-Complete Problems

- They are generally considered in decision form.
- A proposed solution can be verified in polynomial time.



- Every problem in NP can be polynomially reduced to an NP-Complete problem.
- SAT is a fundamental NP-Complete problem.

Fig: NP-Hard and NP-Complete



- SAT
- Hamiltonian Circuit
- Vertex Cover
- Clique

Difference

NP-Hard	NP-Complete
Need not belong to NP	Must belong to NP
May be an optimization problem	Generally considered as decision problem
At least as hard as NP	Both NP and NP-Hard
Need not have polynomial-time verification	Proposed solution can be verified in polynomial time
Example: Knapsack optimization	Example: SAT

Every NP-Complete problem is NP-Hard, but every NP-Hard problem need not be NP-Complete.



8. Explain FIFO Branch and Bound with an example.

FIFO Branch and Bound

FIFO Branch and Bound uses a queue for storing live nodes. FIFO means First-In-First-Out. The node that enters the queue first is selected first for expansion.

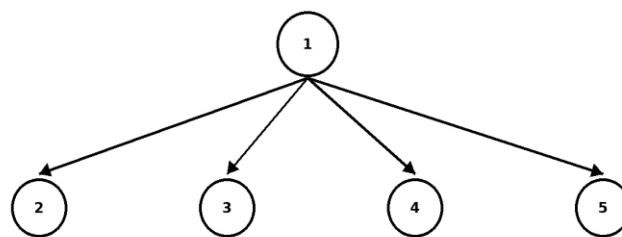
Example – Job Scheduling

Consider $J = \{j_1, j_2, j_3, j_4\}$, $P = \{10, 5, 8, 3\}$, and $d = \{1, 2, 1, 2\}$.

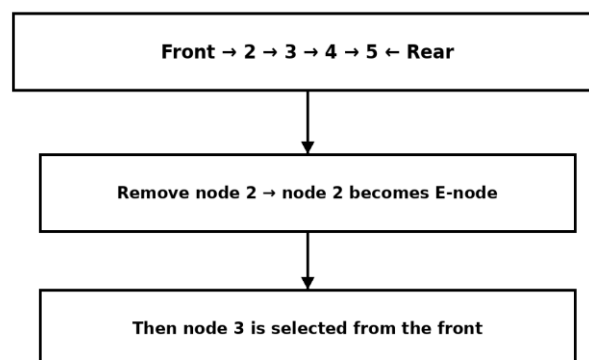
Working Procedure

- Start with the root node 1.
- Expand node 1 and generate its child nodes.
- Insert the generated live nodes into the queue.
- Remove the node at the front of the queue and make it the E-node.
- Generate the children of the E-node and insert promising nodes at the rear.
- Continue the same process until the state-space search is completed.

Fig: FIFO Branch and Bound - Job Scheduling



After expanding node 1



Important Features

- Uses a queue.
- The first live node is expanded first.
- The search proceeds level by level.



- Generated nodes are placed at the rear of the queue.
- It provides a simple systematic search strategy.

FIFO Branch and Bound processes live nodes according to their order of insertion while Branch and Bound cost/bound information is used to control promising search.

9. Explain the general Branch and Bound algorithm. Discuss how live nodes, E-nodes, and dead nodes are handled during the search.

General Branch and Bound Algorithm

Branch and Bound maintains a collection of live nodes and repeatedly selects one live node for expansion. The selection method may be FIFO, LIFO or LC.

1. Live Node

A node that has been generated but is waiting to be expanded is called a live node.

2. E-node

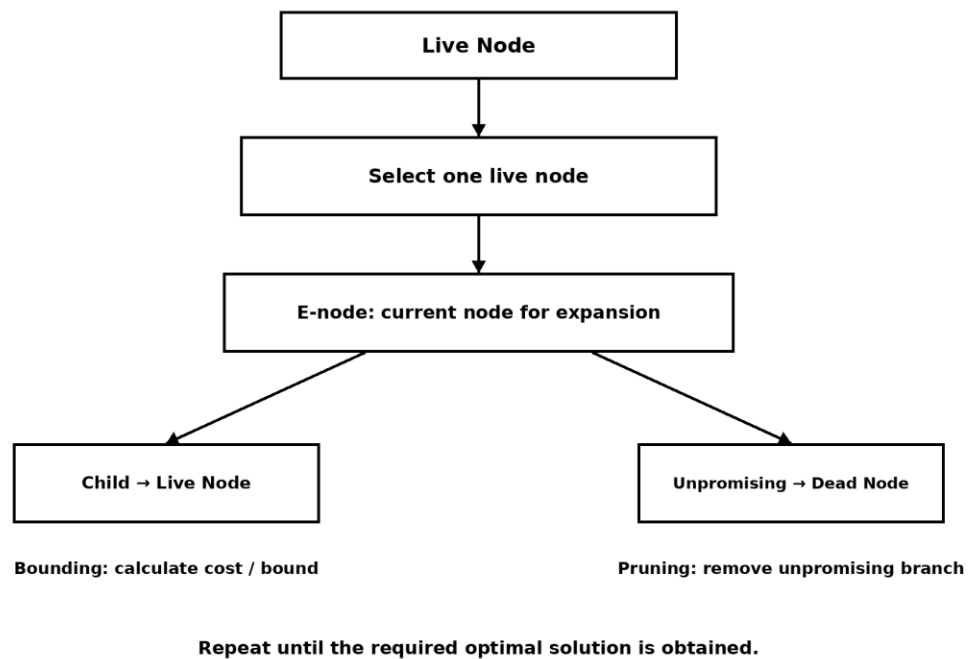
The live node selected for expansion is called the E-node. All children of the E-node are generated before another live node becomes the E-node.

3. Dead Node

A node becomes dead when it is no longer available for further expansion. This may happen after expansion or when the node is found to be unpromising.



Fig: General Branch and Bound - Live, E-node and Dead Node



General Algorithm

Branch_and_Bound()

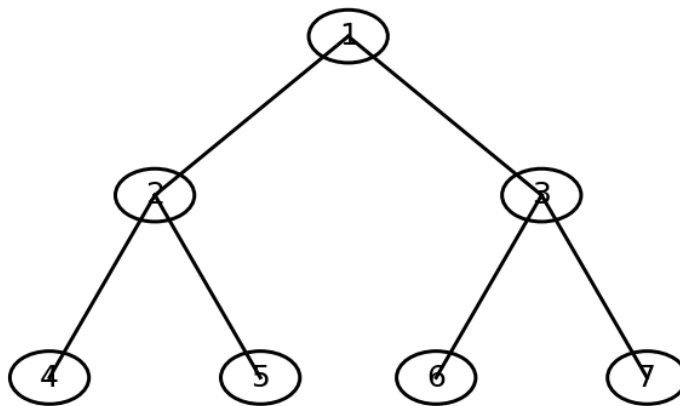
```
{  
    Create the root node;  
    Calculate its cost/bound;  
    Insert root into the live-node list;  
  
    while live nodes exist  
    {  
        Select a live node;  
        Make it the E-node;  
        Generate all children;  
        Calculate cost/bound of children;  
        Discard unpromising nodes;  
        Insert promising nodes into the live-node list;  
    }  
  
    Return the required solution;  
}
```

Handling of Nodes

- The root is initially a live node.



- One live node is selected and becomes the E-node.
- The E-node is expanded and its children are generated.
- Promising children remain live.
- Unpromising children are discarded and become dead.
- The next E-node is selected using FIFO, LIFO or LC strategy.
- The process continues until the required solution is obtained.



State-Space Tree

Selection Strategies

- FIFO – select the first live node.
- LIFO – select the last live node.
- LC – select the least-cost live node.

Efficient Branch and Bound depends on correct management of live nodes, E-nodes and dead nodes together with a suitable cost or bounding function.

10. Compare Backtracking and Branch and Bound.

Backtracking

Backtracking is a problem-solving technique generally used for decision and constraint-based problems. It explores a solution path and returns to an earlier state when the current partial solution cannot produce a valid solution.

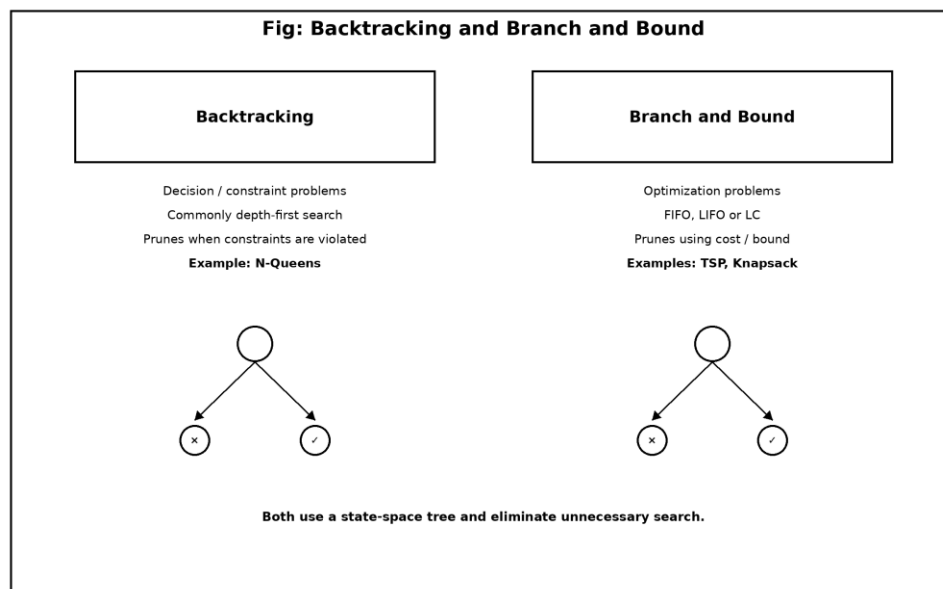
Branch and Bound

Branch and Bound is generally used for optimization problems. It uses a cost or bounding function to determine whether a branch can lead to a better solution.



Comparison

Backtracking	Branch and Bound
Used mainly for decision/constraint problems	Used mainly for optimization problems
Uses depth-first search commonly	Can use FIFO, LIFO or LC
Searches the state-space tree	Searches the state-space tree
Backtracks when constraints are violated	Prunes based on cost/bound
Feasibility is important	Optimality is important
Does not necessarily use a cost function	Uses a bounding/cost function
Example: N-Queens	Examples: TSP and Knapsack
Searches a path and backtracks	Maintains live nodes and selects E-nodes



Similarities

- Both use a state-space tree.
- Both divide the problem into branches.
- Both reduce unnecessary search by pruning.
- Both are useful for difficult combinatorial problems.
- Both systematically explore possible solutions.

TSP Example

The Travelling Salesman Problem can be solved using Branch and Bound to obtain an optimal tour. Backtracking can also explore the state-space tree, but Branch and Bound uses cost/bound information to eliminate unpromising solutions.

Backtracking eliminates a branch mainly when the partial solution violates a constraint, whereas Branch and Bound eliminates a branch when its cost or bound shows that it cannot lead to the required optimal solution.